zühlke
empowering ideas

# AI-Augmented Software Engineering

1-Day Course @ Zühlke

**Zühlke | 12.11.2026 | Zürich**

Public

# Our Coaches

1-Day Course @ Zühlke


**Marco Romanutti**

Lead Software Architect

marco.romanutti@zuehlke.com







**Lukas Grigis**

Lead Software Architect

lukas.grigis@zuehlke.com





**Bulin Shaqiri**

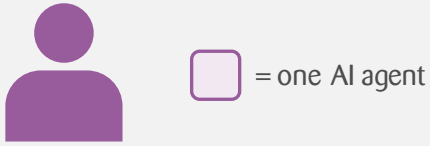
Expert Software Engineer

bulin.shaqiri@zuehlke.com

Public

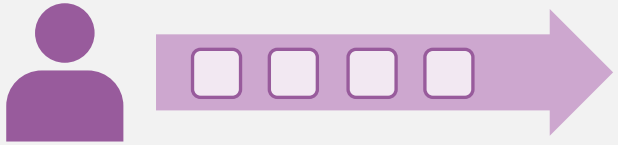
# AI in software engineering takes many forms; our courses teach a methodical way to work with coding agents, from supervised to autonomous

## AI-Augmented Engineering



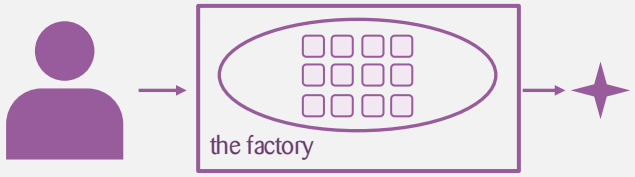

THE HUMAN ROLE
Executes and orchestrates every step

## Agentic Engineering



THE HUMAN ROLE
Defines goals, controls results

## The AI Factory




THE HUMAN ROLE
Sets the target, constraints and governance

## The six levels of AI Autonomy

Fully autonomous begins here

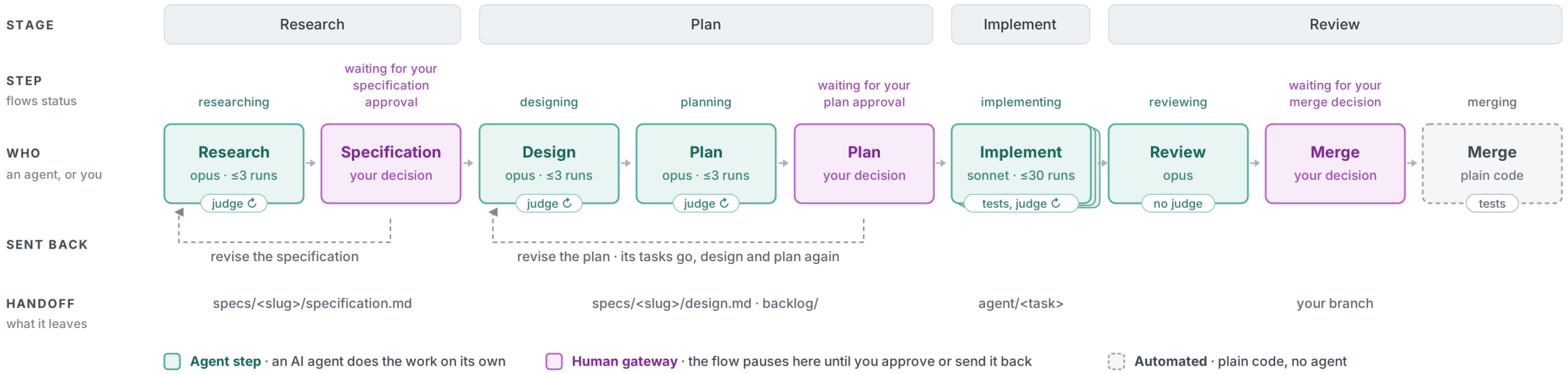
| 01 | 02 | 03 | 04 | 05 | 06 |
|---|---|---|---|---|---|
| Human at the keys | Human in the loop | Human on the loop | Human above the loop | Human sets the goal | You tune the factory, not the software |

AI-Augmented Software Engineering: Foundation

AI-Augmented Software Engineering: Advanced

Public

# Today's concepts let your agents run without constant supervision, while we grant autonomy in a structured, controlled way



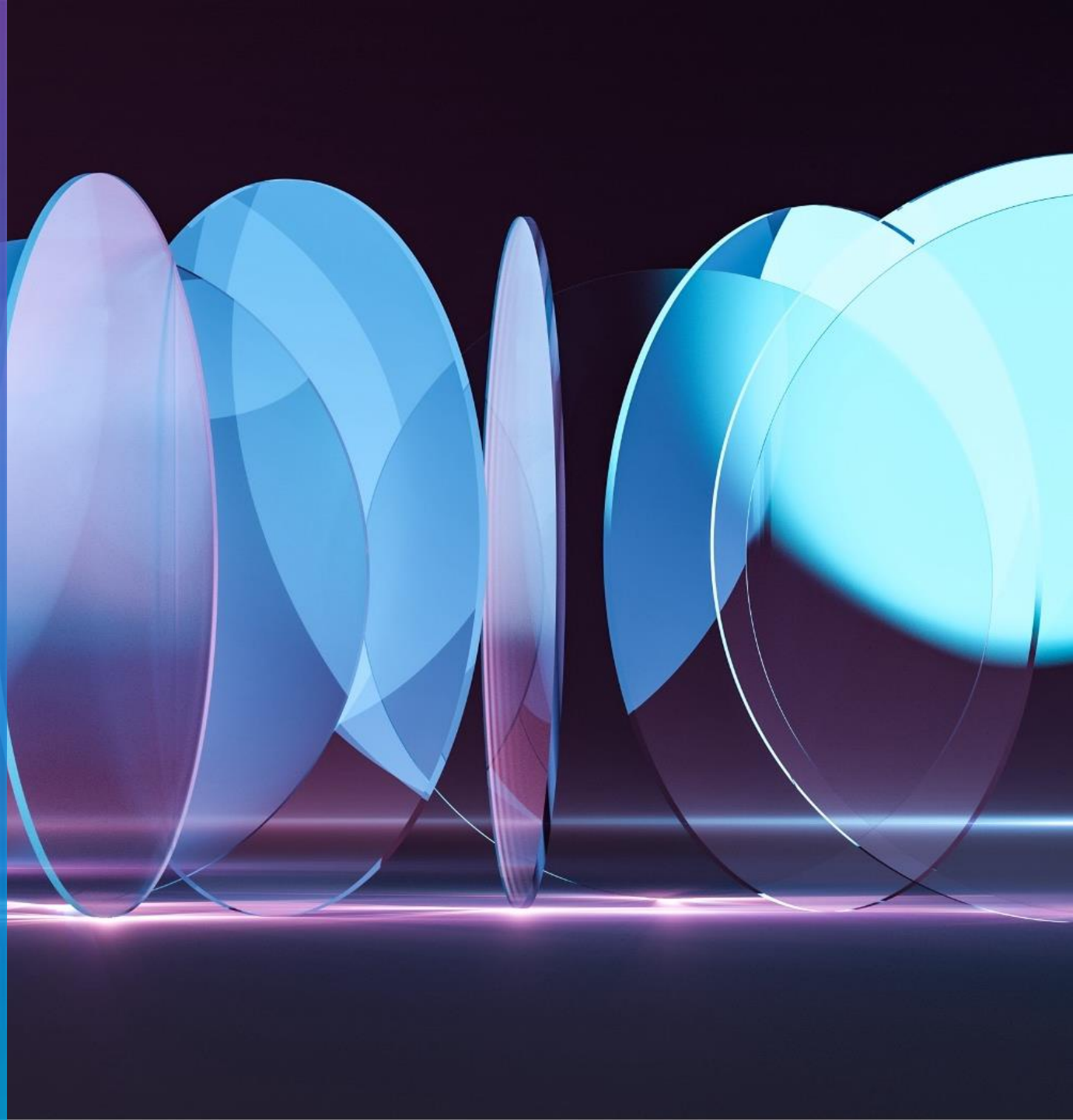


# Recap: Create a common base and refresh the content of day 1

Notes from discussion

© Zühlke 2026

# Sandboxing



Public

# Three layers protect your machine: rules and modes decide whether an action runs, the sandbox limits what it can reach

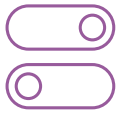


## Permission rules Day 1

**Restrict actions before the agent even runs a command**

Your own list of what the agent may do. Each tool or command is set to allow, ask or deny, and Claude Code checks the list before every tool call.

**Example:** allow npm test, ask before git push, deny reading .env files.



## Permission modes Day 1

**How autonomous you let the agent use commands**

One setting for the whole session that decides how often the agent stops for your approval, from asking before most actions to asking for nothing.

**Example:** autopilot mode (“YOLO”) that skips every approval and runs from start to finish



## Sandboxing

**Reducing blast-radius of running commands**

The operating system fences in shell commands and the processes they start, so they only reach folders and domains you approved.

**Example:** run your agent in a container that blocks network access except to approved hosts.



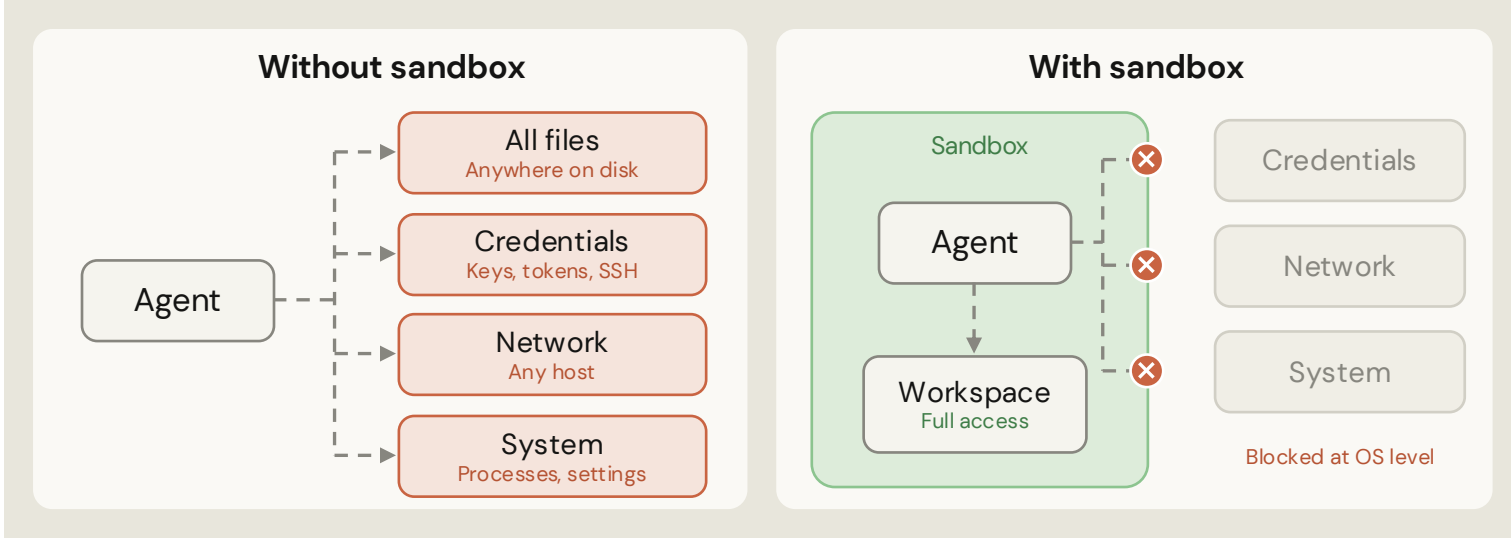
# How can agents run safely unattended on any machine?

## Agent Sandbox

An isolated runtime where AI agents execute code, run tools and access resources safely. Agents run with your credentials, so without a sandbox they can reach anything you can.

It is needed to

- limit the damage if an agent misbehaves
- block access at OS level, not just in a prompt
- let agents run without constant approval



Public

# Isolate by trust: software isolation for daily work, hardware isolation for untrusted code

| Approach | What's sandboxed | Use case | Examples |
|---|---|---|---|
| Sandboxed shell tool | Shell commands the agent runs. Coverage of file edits and MCP servers varies by tool | Everyday work with fewer permission prompts. Check what stays outside before running unattended | Claude Code /sandbox<br>Copilot CLI /sandbox<br>Codex CLI --sandbox |
| Agent process sandbox | The whole agent process, incl. file tools, MCP servers and hooks. Shares your OS | Unattended runs without Docker, using OS primitives (bubblewrap, Seatbelt) | Anthropic sandbox-runtime (beta) |
| Container (dev or custom) | Agent and toolchain with their own filesystem and network. Shares the host kernel | Unattended runs with a default-deny egress firewall. One standard setup for the team | **Zühlke AI Coding Sandbox**<br>Dev Containers spec<br>Claude Code dev container |
| Virtual machine, microVM | Agent with its own operating system and kernel | Untrusted code, strict security policies, agents that need to run Docker themselves | Docker Sandboxes (sbx, microVM) |
| Cloud sandbox | Agent with its own operating system and kernel, hosted by a provider | Untrusted code without local setup. Your code runs on the provider's infrastructure | Claude Code cloud sessions<br>Copilot CLI --cloud<br>E2B |

**Software isolation**
OS sandboxes, containers
- Shared host kernel
- Light and fast, enough for daily work

**Hardware isolation**
VMs, microVMs
- Own kernel per sandbox
- For untrusted code

**No sandbox is a hard guarantee:** anything the agent can read can still reach a host you allow, and a writable project folder can still be damaged.



# LAB: Sandboxing agents

## Context

So far the agent worked directly in your checkout, and you approved every command.

A sandbox adds a boundary around the agent: even if it goes wrong or gets compromised, it can't touch your machine, files or network beyond its own API.

In this lab you'll turn on the sandbox, check whether the agent can still reach the internet, block all outbound network access, and check again.

Day 2 needs the dev container from this branch: rebuild it once, or open a Codespace on `lab/sandboxing`.

## Your Task

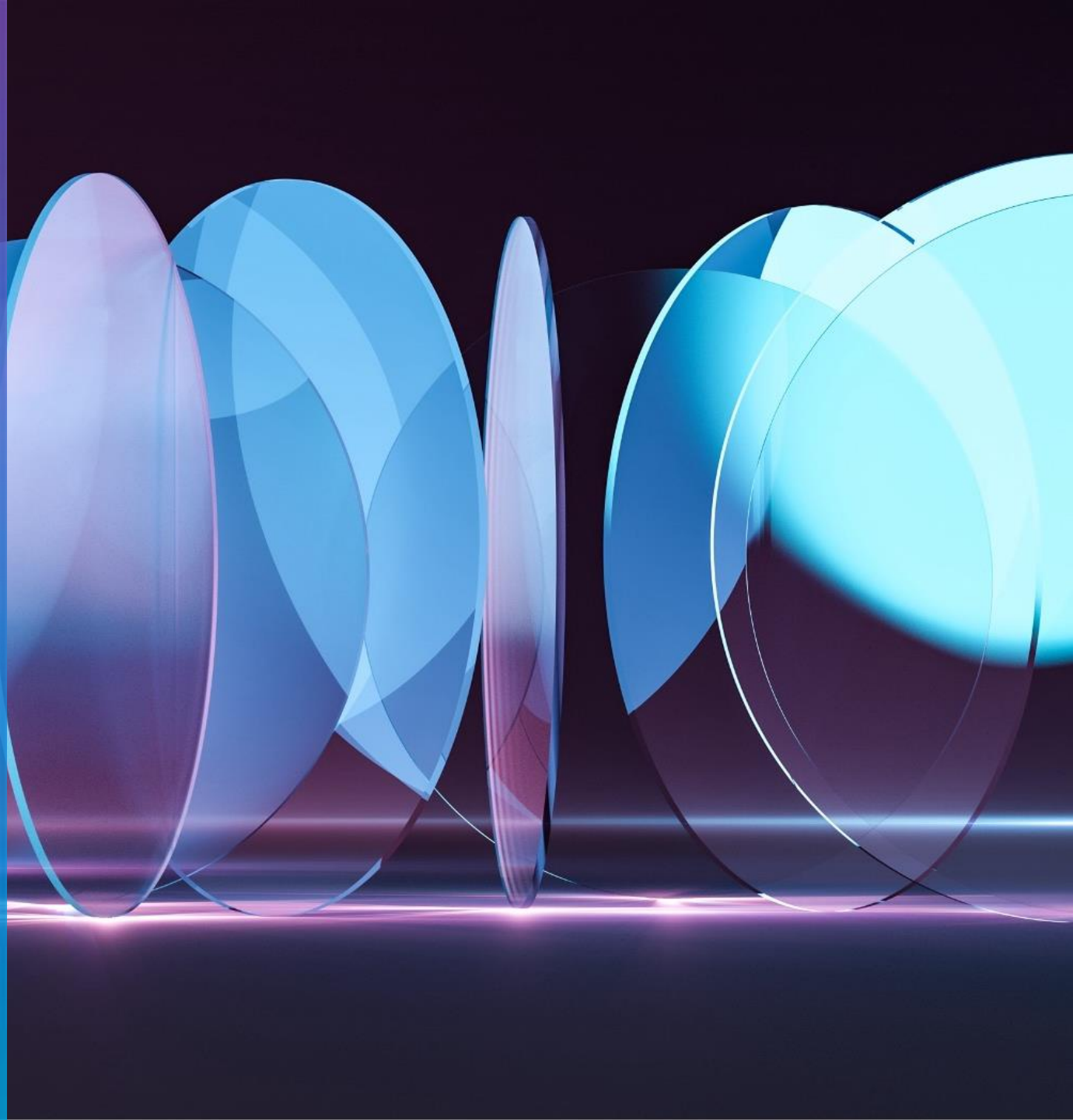
Start the lab setup with the following command:

```
task lab/sandboxing
```

Choose **GitHub Copilot CLI** or **Claude Code** and use it for every step:

- enable the sandbox,
- check that `curl` still reaches the internet,
- block all outbound network access,
- check again, then turn the network back on.

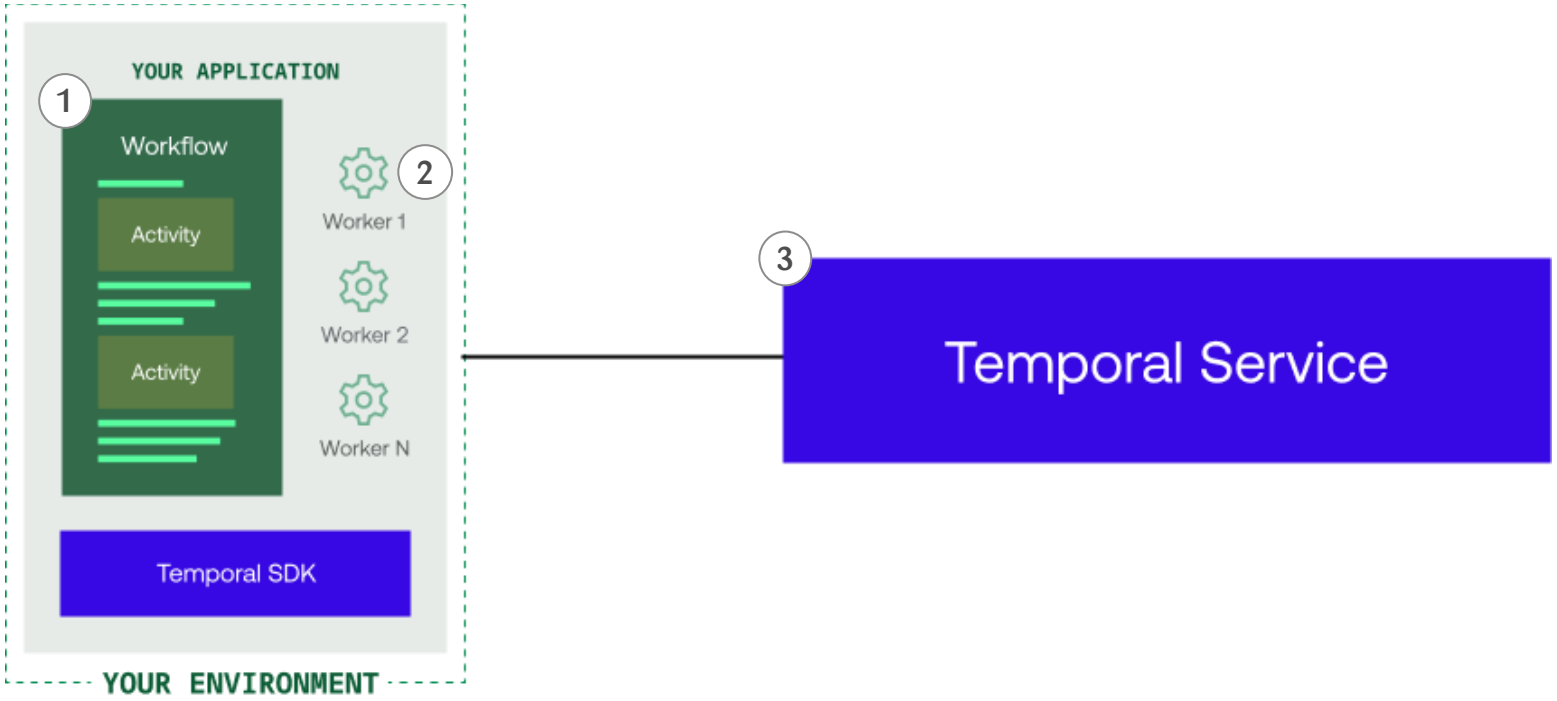
# Durable Workflows



Public

# Durable workflows let agent runs survive crashes and resume where they stopped

We use Temporal as the orchestrator for our agents: workflows define the steps, workers execute them and the Temporal Service keeps every run durable.




**1 Workflow**

Orchestration logic as ordinary code. Temporal records each step, so after a crash the run **replays its history and continues.**

**2 Workers**

Stateless processes that run workflow and activity code. If one dies, **another worker picks up its tasks**; add more to scale out.

**3 Temporal Service**

Persists the state and event history of every run, so **nothing is lost when a worker fails.** We run the open source version locally.

*Source: Temporal Technologies, temporal.io documentation*



# LAB: Durable Workflows

## Context

From here on, agents mostly run unattended. The harness in **.harness/** starts them and runs on Temporal, so after a crash the work goes on where it stopped.

This lab gets you comfortable with Temporal and the **flows** commands you'll use in all upcoming labs.

## Your Task

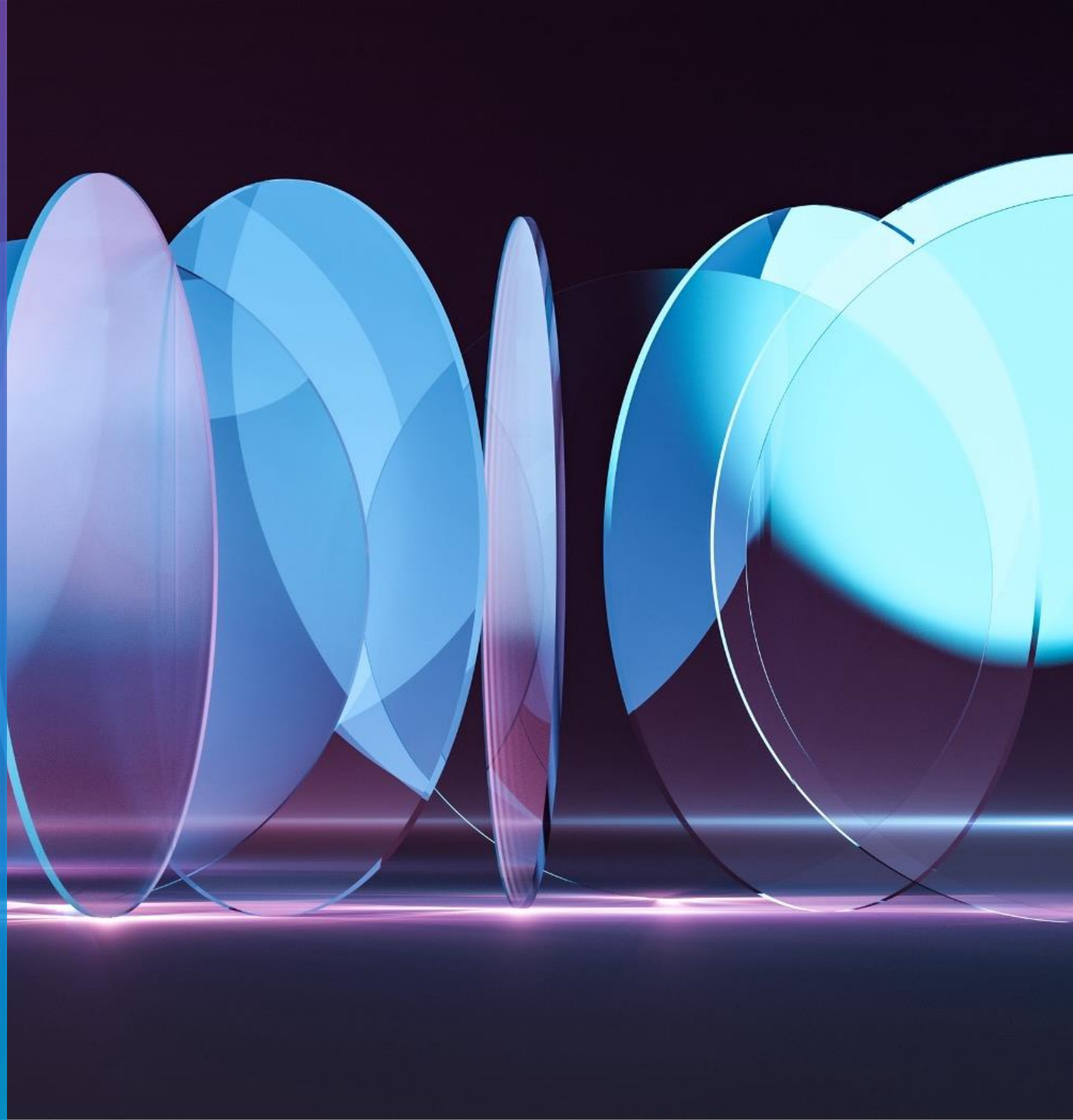
Run **task lab/durable-workflows**

- Start Temporal and the worker: **task flows:start**
- Open **task flows:watch** on localhost:8234
- Start the pipeline with a feature request; the research agent writes a specification
- Don't approve it: stop and restart Temporal, and check that it still waits

## Tipps & Tricks

- Commit or stash first: the research agent only sees committed work.
- The Temporal UI on localhost:8233 shows every finished step of the pipeline.
- In Codespaces, open a new Codespace on **lab/durable-workflows**.

# Tracer Bullets & good slicing





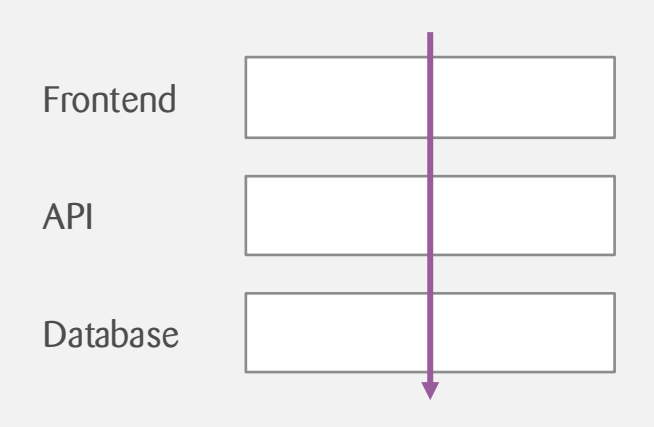
# What could be the issue when giving agent huge tasks to fulfil?

Notes from discussion

© Zühlke 2026

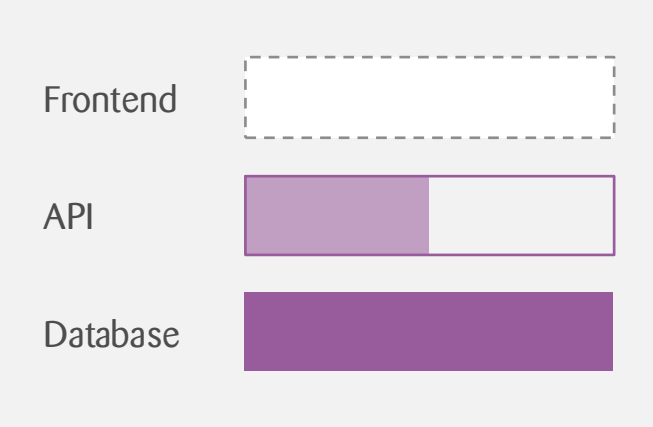
Public

# Tracer bullets: tiny, working end-to-end vertical slice of a feature that touches every system layer to validate functionality and give agents early feedback



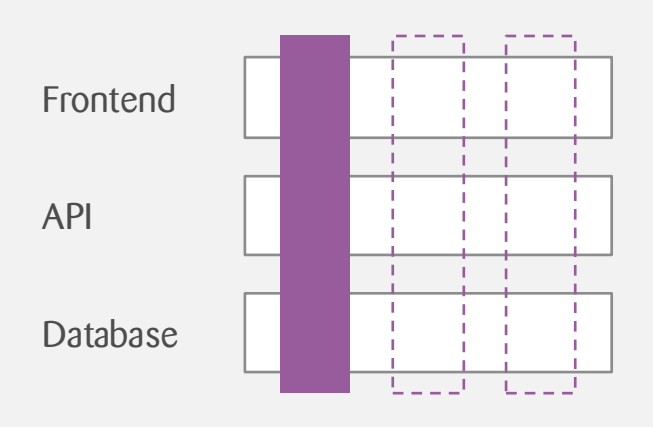

## 1) Systems have layers

A system consists of separate layers such as database, API and frontend. The critical path runs through all of them. AI agents tend to build a lot of code before checking whether this path actually works.




## 2) Agents build layer by layer

Agents usually work horizontally: first the API endpoints, then services, persistence and finally the UI. Each layer looks fine on its own, but the parts only come together at the end. Wrong assumptions stay hidden for a long time and lead to heavy rework.




## 3) Tracer bullets cut vertically

Instead, build one small slice of functionality through every layer the critical path needs. This tests architecture, integration and assumptions early, and you have working software to show from the start.



# LAB: Tracer Bullets

## Context

A specification says what should happen, not how to split it into tasks. By hand, blockers get missed and work gets split by layer.

The **plan** skill first lists the blockers, then slices the rest into vertical tracer-bullet tasks that go through every layer they touch.

## Your Task

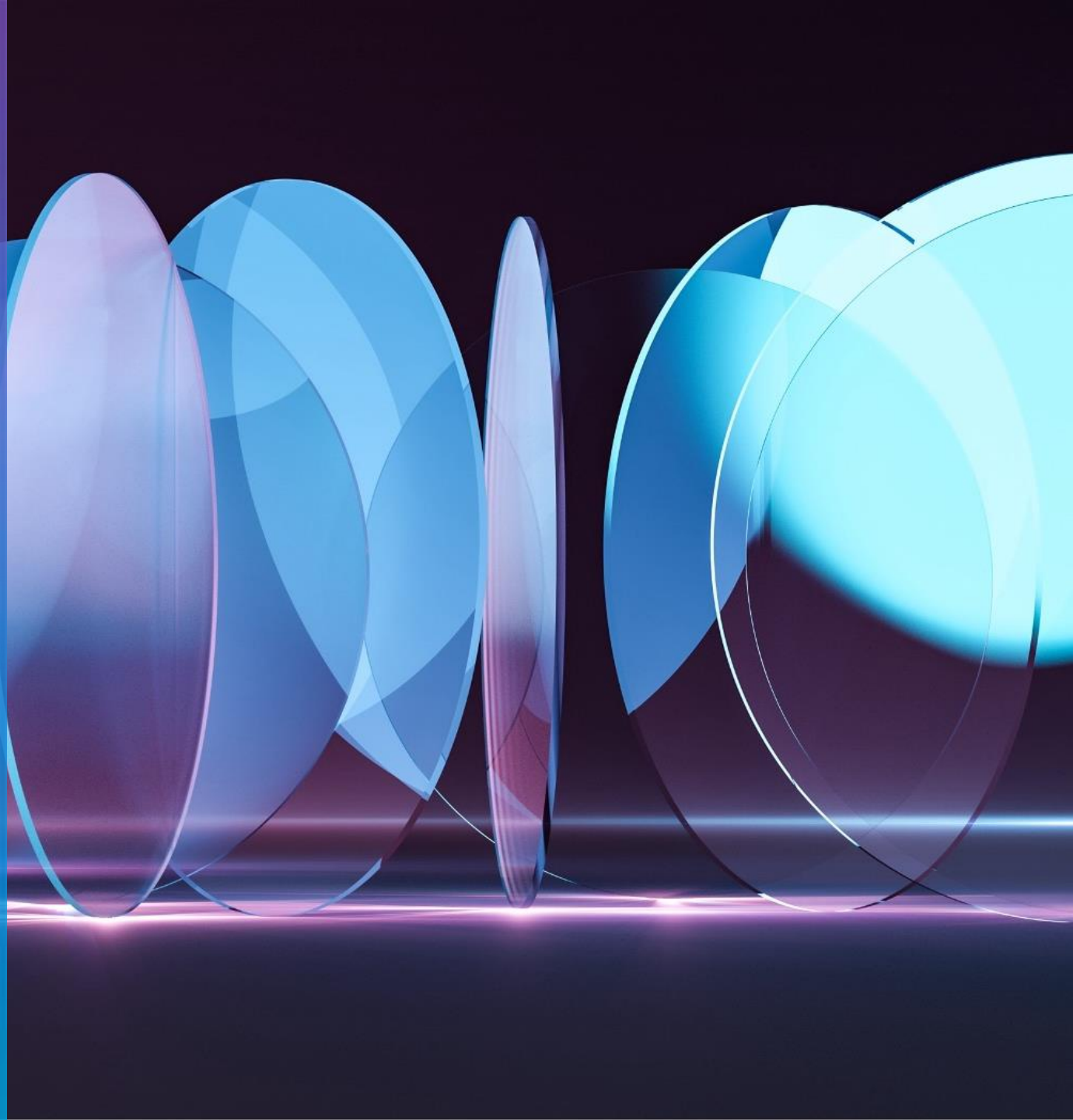
Run **task lab/tracer-bullets**

- Invoke the skill: **/plan flight-delays-dashboard**
- The specification and design are already in **specs/flight-delays-dashboard/**
- Review the tasks it writes in **backlog/**

## Tipps & Tricks

- Does each task go through every layer it touches?
- Can you demo or verify each task on its own?
- Is prefactoring its own task, ordered first?
- Is a task only ready when nothing blocks it?

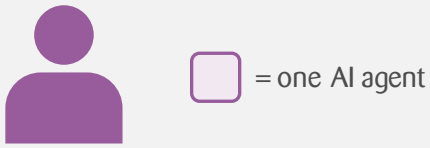
# Autonomous Agents



Public

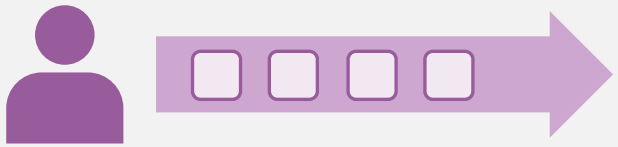
# AI autonomy level can be individual per team and application, governed by its risk profile

## AI-Augmented Engineering



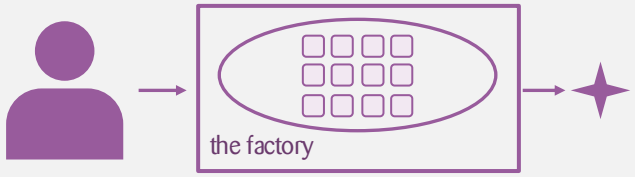

THE HUMAN ROLE
Executes and orchestrates every step

## Agentic Engineering



THE HUMAN ROLE
Defines goals, controls results

## The AI Factory




THE HUMAN ROLE
Sets the target, constraints and governance

## The six levels of AI Autonomy

Fully autonomous begins here

| 01 | 02 | 03 | 04 | 05 | 06 |
|---|---|---|---|---|---|
| Human at the keys | Human in the loop | Human on the loop | Human above the loop | Human sets the goal | You tune the factory, not the software |



# What are issues of long-running agentic conversations?

Notes from discussion

© Zühlke 2026

Public

# Ralph loop: a fresh context every run keeps quality stable

**Long agent session**

One context collects every old attempt




**Ralph loop**

Same prompt, fresh context every run




.harness/progress.md

```
# Task: password reset
[x] AC1  reset link sent by email
[x] AC2  link expires after 1 h
[ ] AC3  new password validated                    next run
[ ] AC4  audit log entry written
Notes: AC2 needed DB migration 014
```

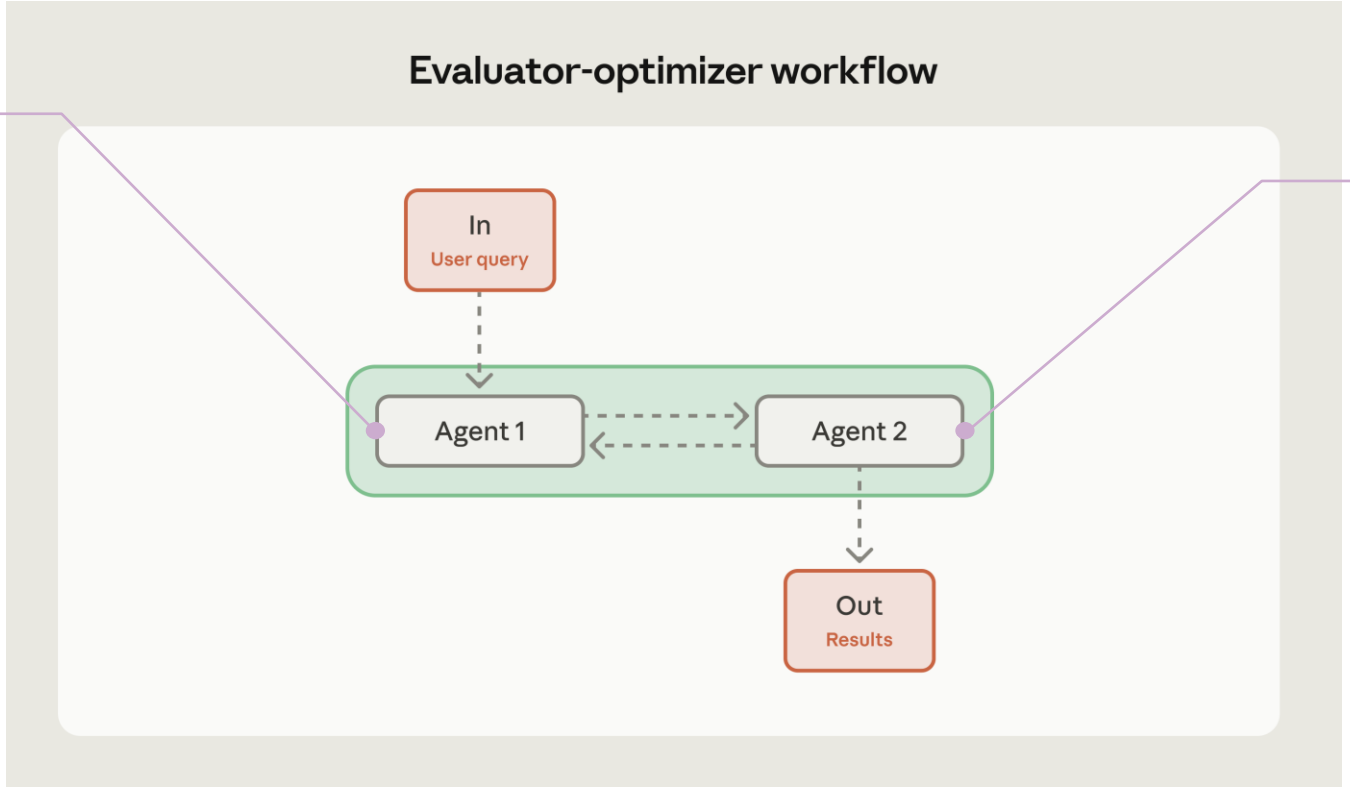
**The agent's only memory**

Each run reads the file at the start, does the next open step and ticks it off. The git history keeps the commits.

Public

# Generator-evaluator pattern: a stronger, independent agent judges the outpout from a previous agent

**Agent 1: generator**
Each run follows the generator and validator pattern: the agent generates the step, a judge validates it. The implementor runs on Sonnet.




**Agent 2: validator**
The judge checks the step: first plain code (did it commit, do the tests pass, did it leave the skills and the harness alone?), then a read-only agent with the judge skill. A rejected step goes back to the next run as feedback. The judge runs on Opus (on GPT-6.1 Sol with Copilot): a weaker judge would approve bad work.



# LAB: Ralph Loop

## Context

Long agent sessions get worse as the context fills with old attempts. A Ralph loop runs the same prompt again with a fresh context until the task is done.

Each run takes one small step, tested and committed. A judge on a stronger model checks it; **progress.md** and git history are the memory.

## Your Task

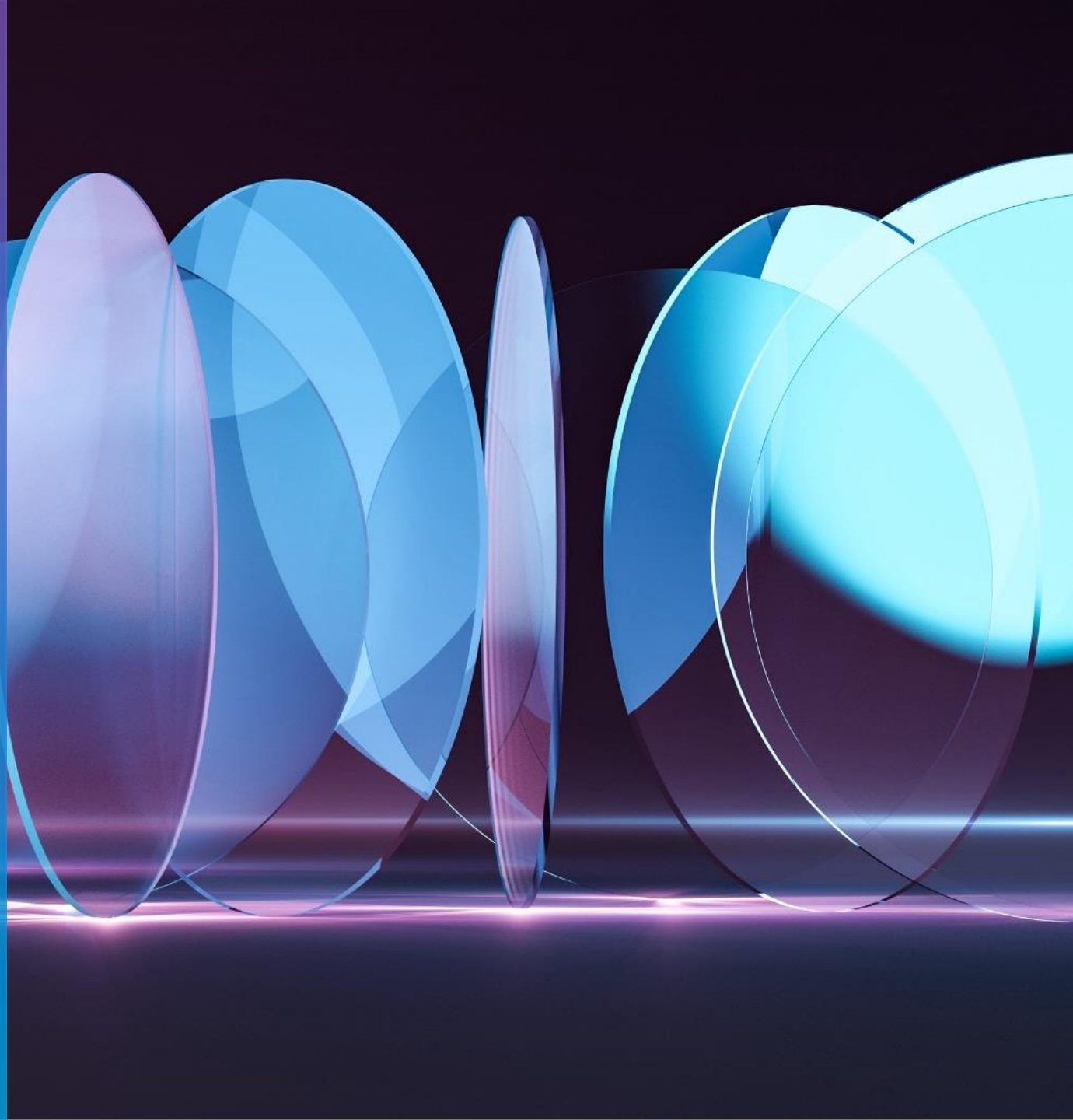
Run **task lab/ralph-loop**, then **task flows:start** and **task flows:watch**

- Do one run on TASK-1, check its progress, then throw it away
- Loop TASK-1 until done
- Merge it, then run **task test** and read **learnings.md**

## Tipps & Tricks

- The agent runs unattended: never run it without the sandbox.
- If the loop waits for you, click Answer and send a decision.
- Review **.harness/learnings.md** like code: every run reads it.

# Paralellizing Work





# What could go wrong when running multiple agents in parallel?

Notes from discussion

© Zühlke 2026



Demo

# Git Worktree



# LAB: Parallelizing work

## Context

Two agents in one checkout get in each other's way: same files, same branch, broken tests.

A git worktree gives each agent its own working directory and branch, so they work at the same time and you merge the results like any branch.

## Your Task

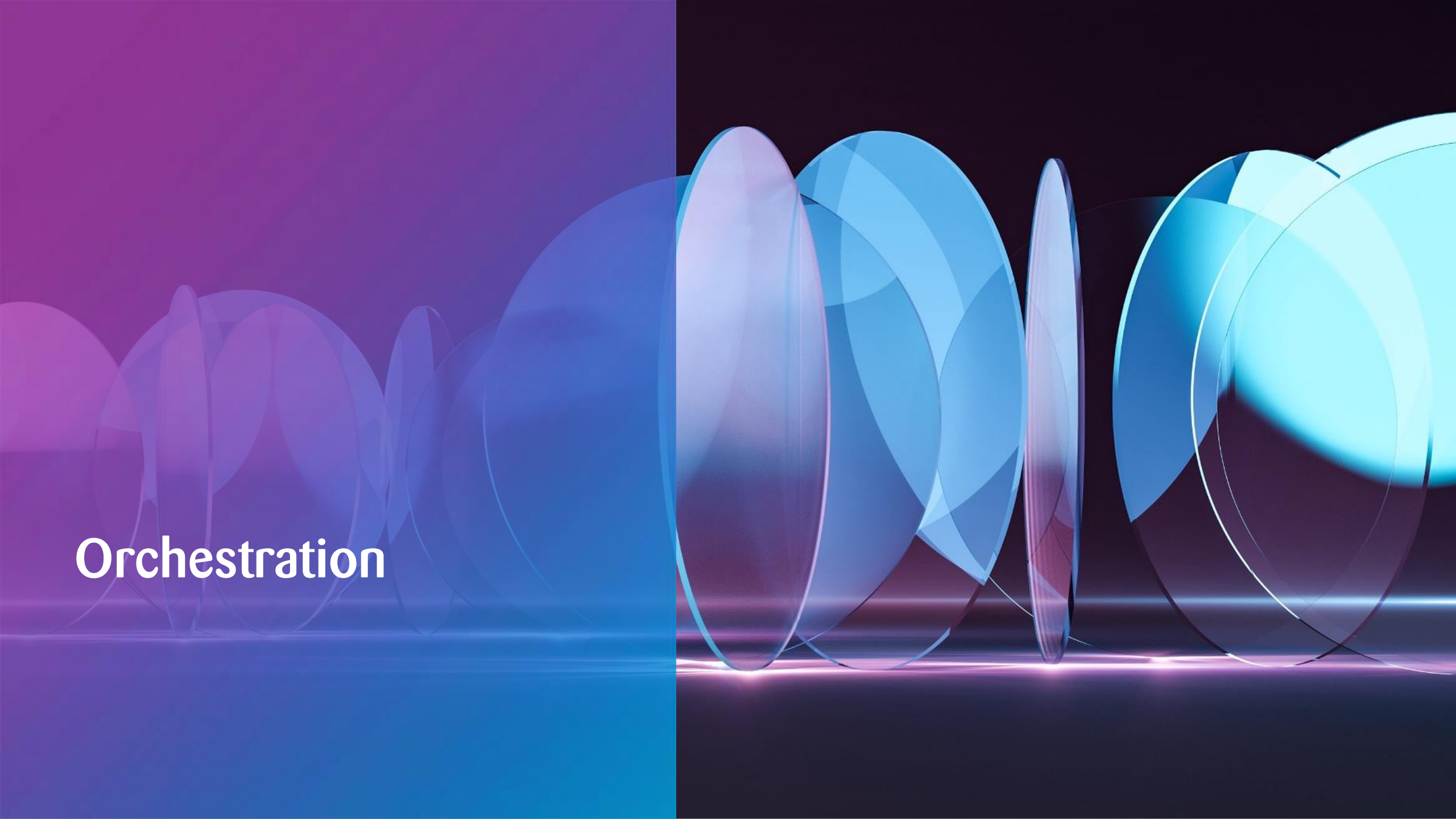
Run **task lab/parallelizing-work**, then **task flows:start** and **task flows:watch**

- Loop TASK-2 and TASK-3 until done, at the same time
- Find the worktrees: **git worktree list**
- Run the app from the TASK-2 worktree, then throw both loops away

## Tipps & Tricks

- Worktrees only separate files: ports, databases and caches are shared.
- Port 8080 in use? Start with **SERVER_PORT=8081**.
- Each loop has its own **.harness/progress.md**.

# Orchestration





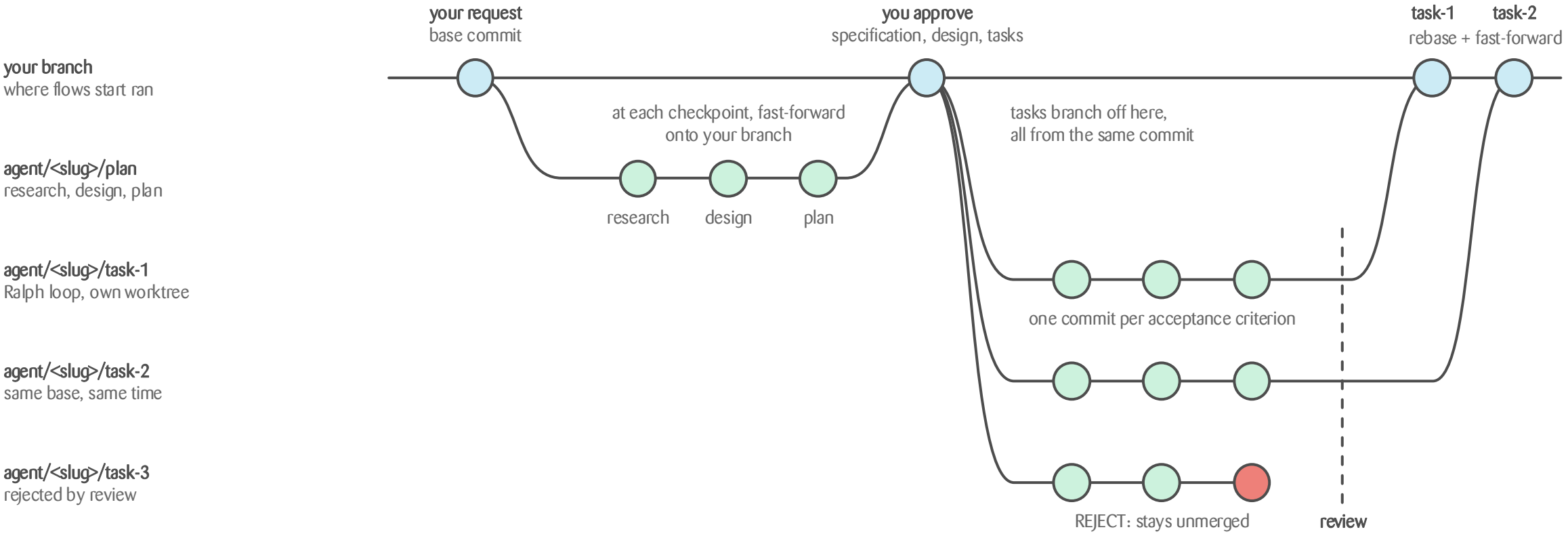
# How would you draw the git structure of an agentic development workflow?

Notes from discussion

© Zühlke 2026

Public

# Each task runs in its own worktree, so agents work in parallel without touching your branch




Blue circles are commits on your branch, green ones are agent commits, red is a branch the review rejected. The dashed line is the review, the only gate before a merge.



# LAB: Orchestration

## Context

This lab runs the whole pipeline: research, design, plan, one Ralph loop per task, review and merge.

Each phase has its own judge. You decide at three checkpoints: the specification, the plan and the merge.

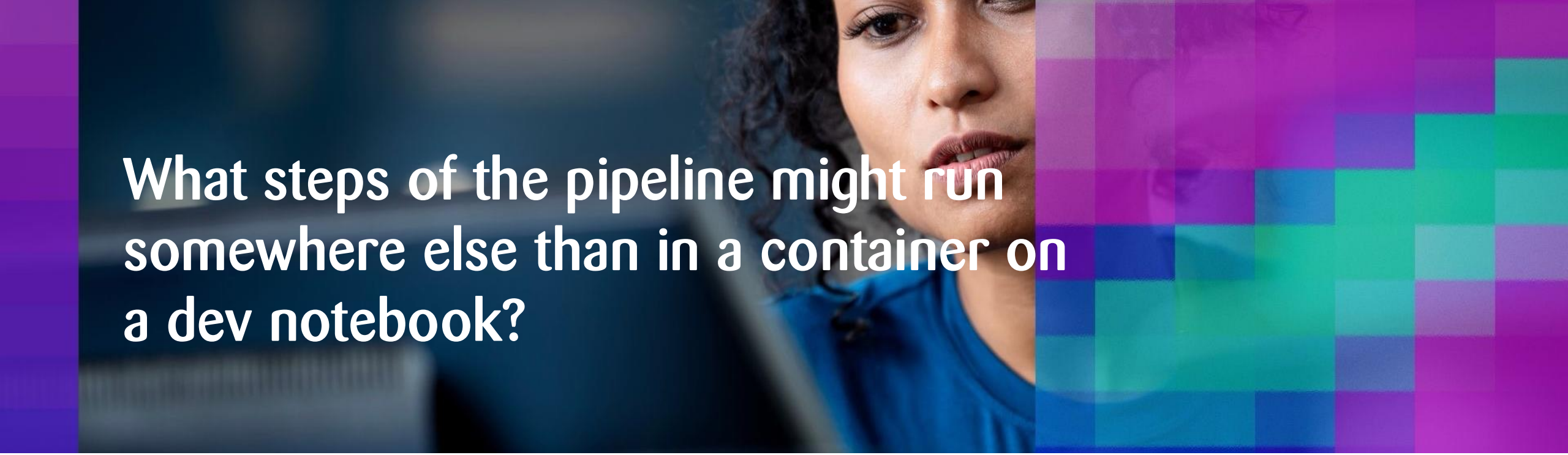
## Your Task

Run **task lab/orchestration**, then **task flows:start** and **task flows:watch**

- Start the pipeline with the feature request
- Approve the specification, then the plan and its tasks
- Merge after the review, then check the dashboard with **task run**

## Tipps & Tricks

- Not right? **Send back**… with feedback and the agents try again.
- A failing loop doesn't stop the others.
- Clean up with **flows cleanup**.



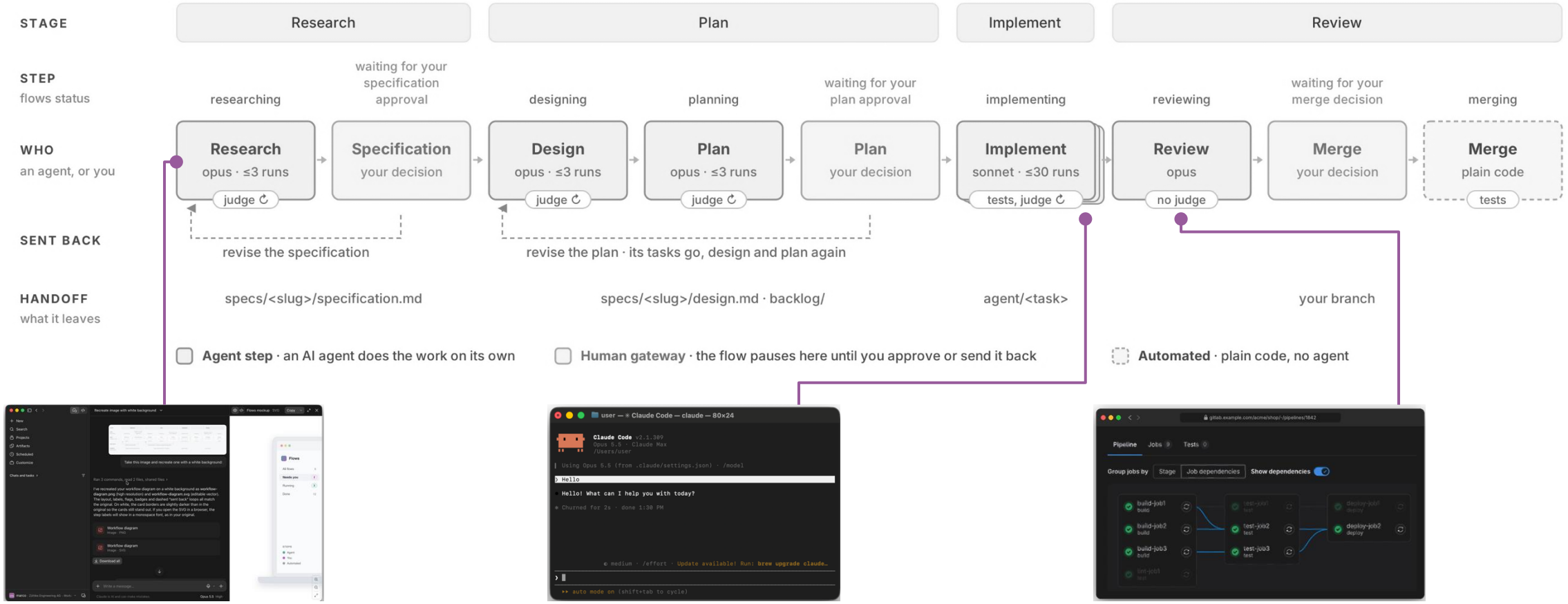
# What steps of the pipeline might run somewhere else than in a container on a dev notebook?

Notes from discussion

© Zühlke 2026

Public

# No one-size-fits-all: Design the agent pipeline around your team: each role gets its phases, tools and gates






**Research:** BAs research and specify features in desktop app

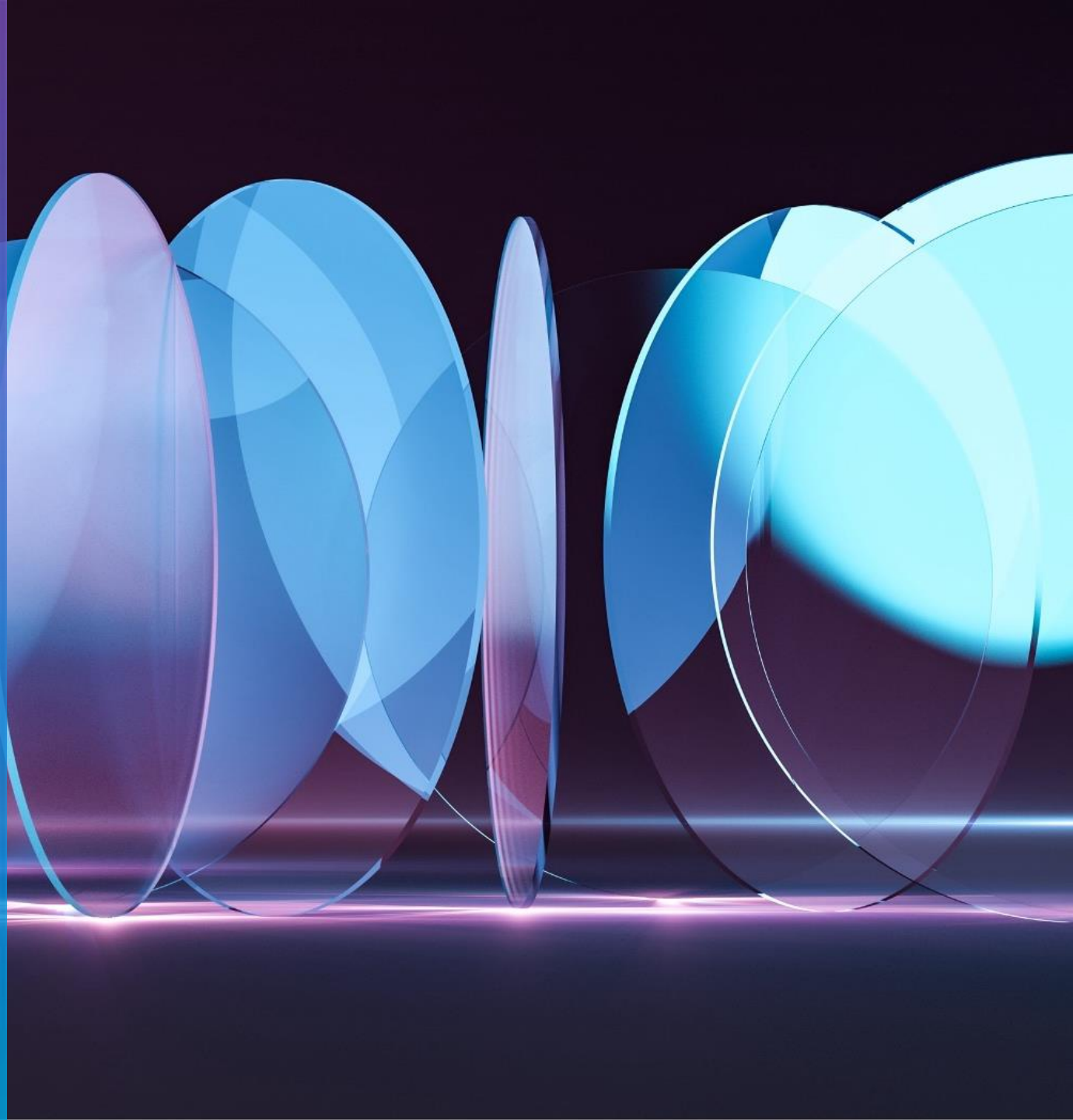



**Implement:** Developers run and steer agents from the terminal




**Review:** Automated code review in the CI/CD pipeline on every MR

# Evals





# How do we ensure our harness still works after a change?

SKILL.md · v1

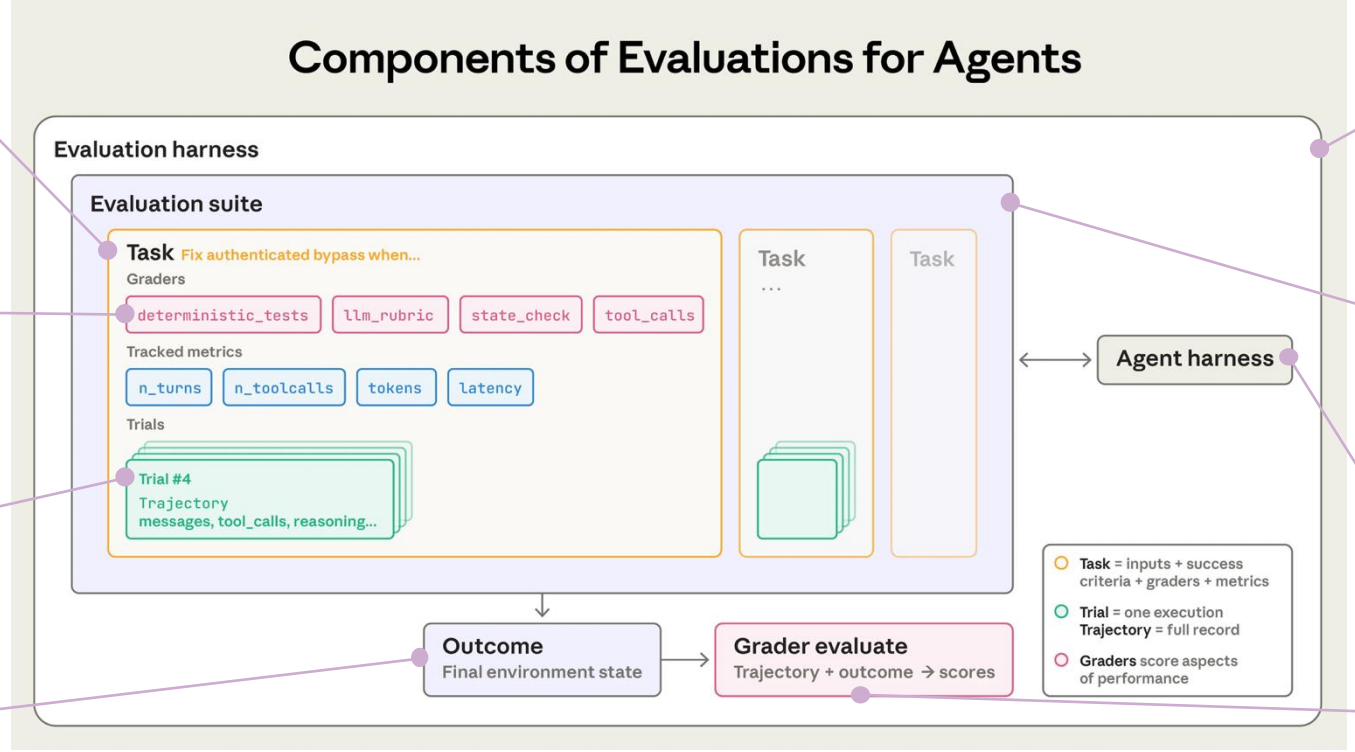
```
---
name: code-review
- description: Review my code
---
## Overview
Reviews a diff and lists issues.
```

SKILL.md · v2

```
---
name: code-review
+ description: Review a PR diff for bugs,
+   missing tests and security issues.
+   Use when asked to review a change.
---
## Overview
Reviews a diff and lists issues.
```

Public

# Evaluations (evals) tests the agent harness: they runs tasks, records each trial and grades the outcome




**Task**
One job with clear success criteria

**Grader**
Deterministic and non deterministic checks that are applied for a specific task

**Trial**
One attempt at a task

**Outcome**
Final state of the environment

**Evaluation harness**
Runs trials, records them, applies graders

**Eval suite**
A collection of representative tasks

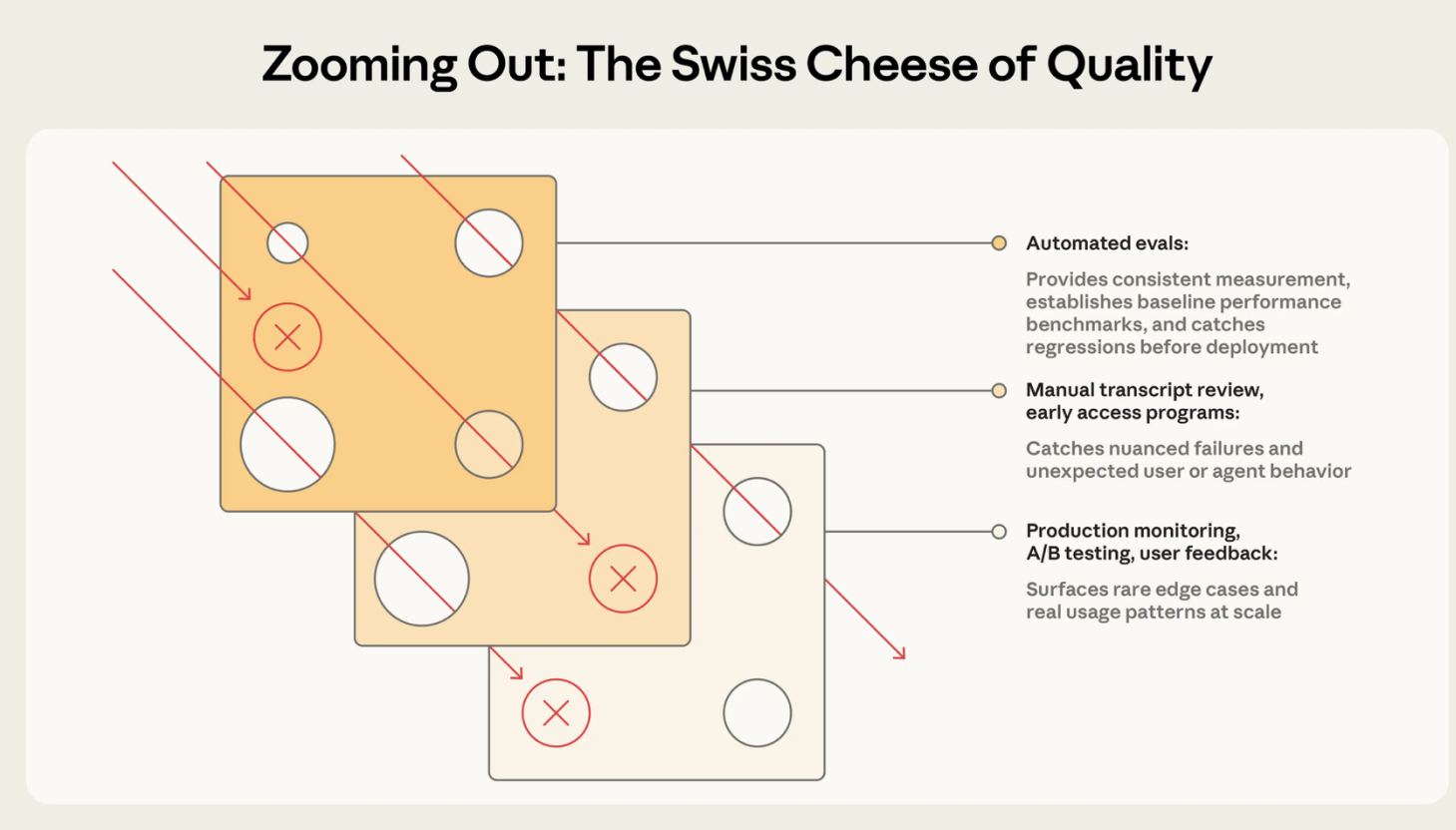
**Agent harness**
System under tests

**Grader evaluate**
Verdict, if the task passed or failed

*Source: Anthropic, "Demystifying evals for AI agents", January 2026*

Public

# Agents behave differently on every run, so no single quality layer catches every failure and you have to run evals multiple times  to get a clear verdicts

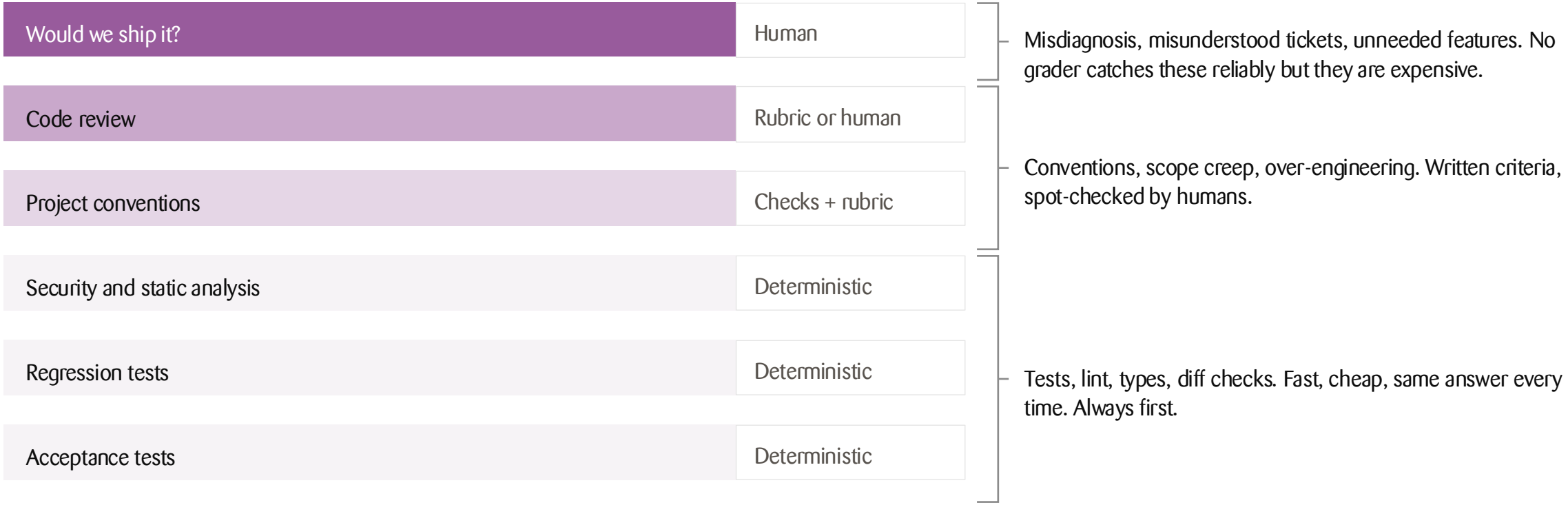



*Source: Anthropic, "Demystifying evals for AI agents", January 2026*

Public

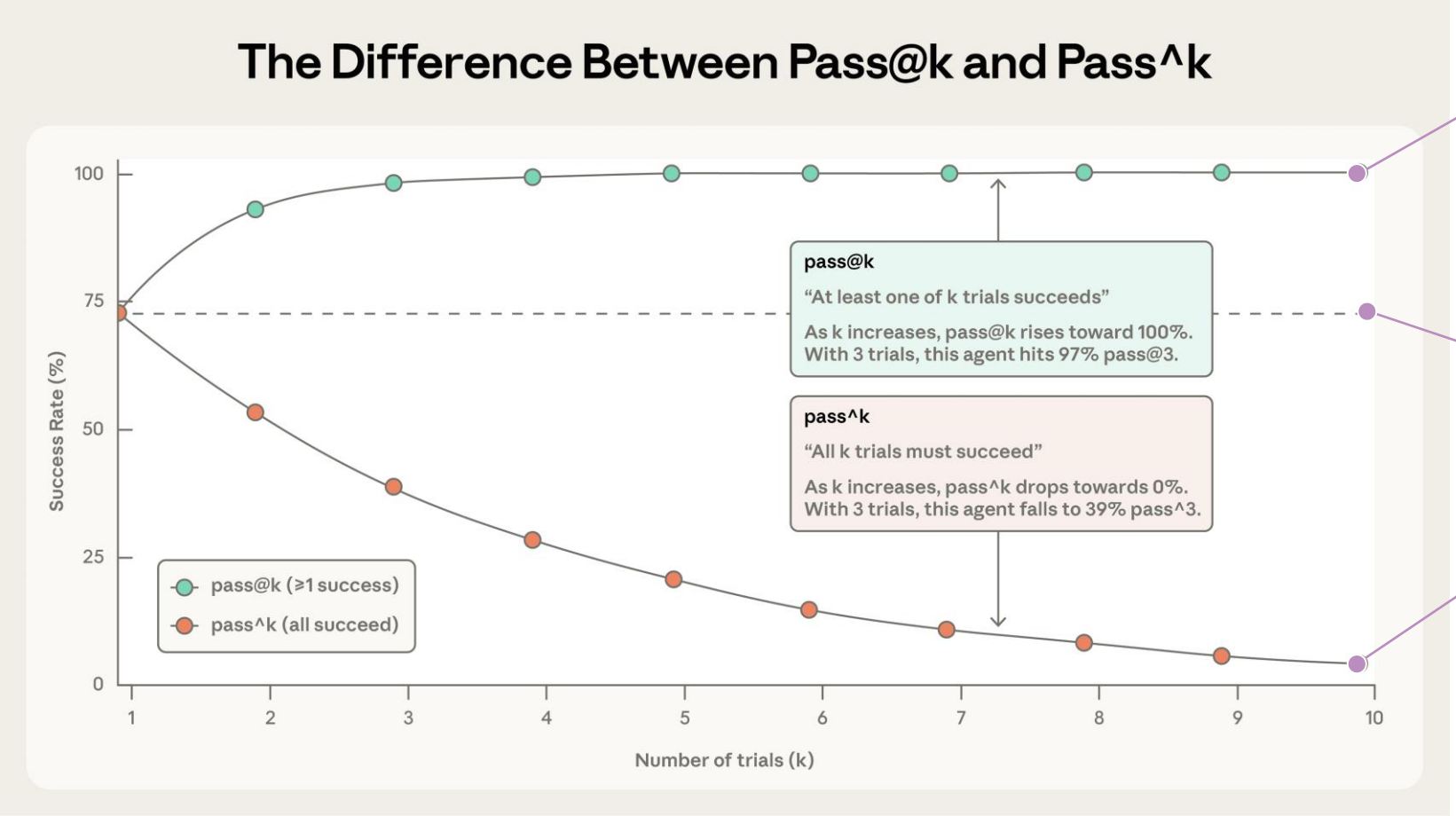
# Stack your quality gates: deterministic at the base, human judgment at the top

Quality gate, top to bottom




**Your test become graders:** Deterministic first: fast, cheap, reproducible. An LLM judge or a human only where judgment is needed. The eval adds what tests lack: a task bank, repeated runs, a before/after comparison.

# Pass@k tests what an agent can do while Pass^k tests whether you can rely on it




**pass@k: one success is enough**
Use when picking the best of several attempts is fine, e.g. a coding assistant proposing fixes.

**At k = 1 they are identical**
Both equal the per-trial success rate.

**pass^k: every run must succeed**
Use for customer-facing agents that must work reliably every time.

*Source: Anthropic, "Demystifying evals for AI agents", January 2026*



Demo

# Evals with promptfoo



# LAB 13: Evals

## Context

Agents fail quietly: change a skill's description and the agent may stop loading it, or load it for everything.

Evals are the tests of your harness. Here you change a skill's description and check with promptfoo that it still loads when it should, and only then.

## Your Task

Run **task lab/evals**

- Run the skill-loaded eval: **task evals:skills**
- Make the **testing** description too broad and rerun it
- Write a better description until every case passes

## Tipps & Tricks

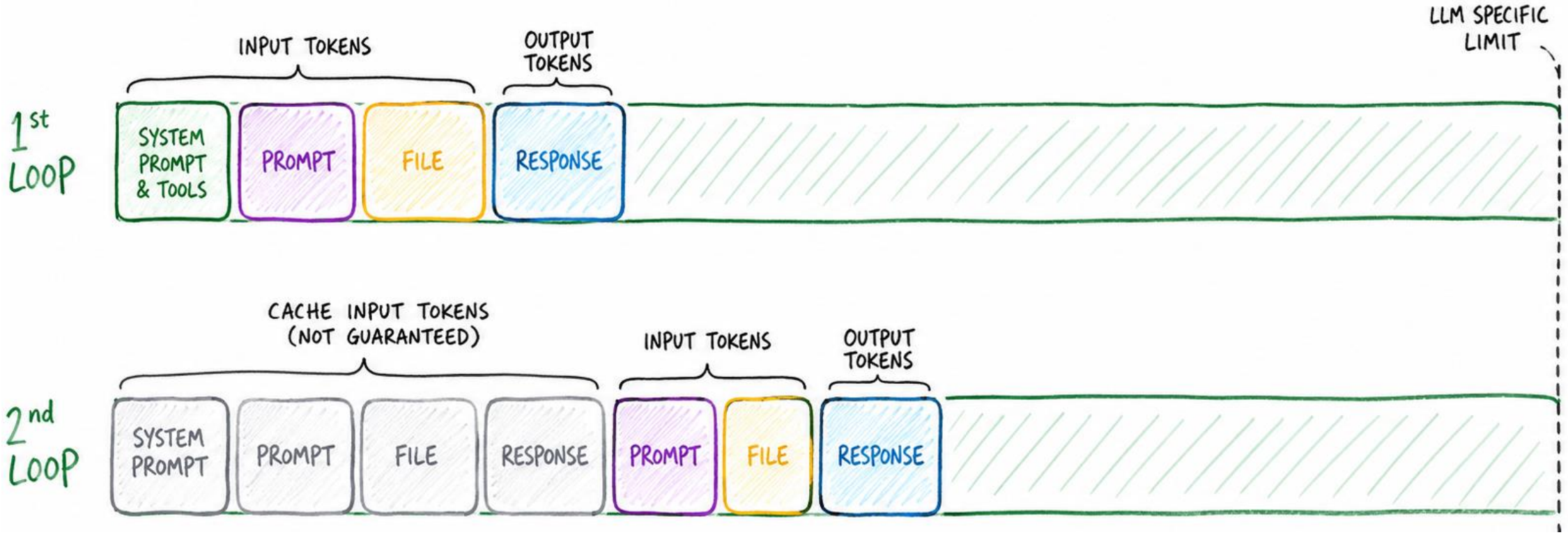
- Only pass^k cases, right on every run, are reliable.
- Rerun one skill:
  **task evals:skills -- --filter-pattern '^testing'**
- See results with **task evals:view** on localhost:15500.

# Token Economics



Public

# To own token allowance effectively, we have to understand the different types of tokens: input tokens, cached tokens, and output tokens




**Input tokens** - what you send to the model **Cached tokens** - context the model uses from previous interactions **Output tokens** - what the model generates in response.

*Source: https://www.youtube.com/watch?v=0IOcPqubpMc*



# How can I get the most out of my Token allowance?

### Provide more clear specification of what to do

If you provide a vague prompt, the model has to do a lot of work to figure out the necessary context and the target. This means it will perform a lot of file searches, read files that may be unrelated to the tasks, and may have to backtrack a lot. All of this costs a lot of tokens.

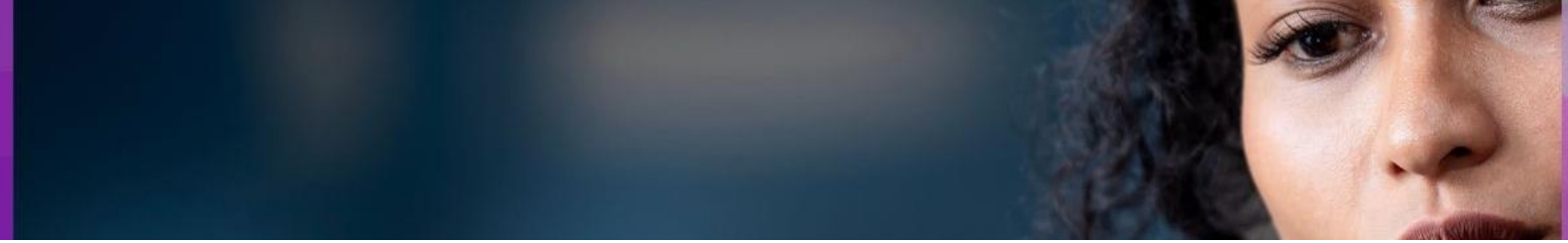
If instead you provide a clear task with hints on what input to use and how to verify the result, the agent will have to process much less data.

### Use Structured Workflows

By following a structured workflow, you significantly increase the chance that the agent will get a task right on the first attempt, preventing costly rework.

### Use the right model for the right job

Not every task needs the most capable model. Learn when to use GPT-5 mini or GPT-5.2 instead of GPT-5.5 and when to use Claude Sonnet over Claude Opus.



### Instruct the model to generate less output tokens

Output tokens are much more expensive than input tokens. By instructing the model to be more concise, for example, using a caveman skill, you can significantly reduce your token spend.

### Use code for deterministic tasks, and only use LLMs where their strengths matter

Take advantage of deterministic tools like scripts, linters, tests to provide guardrails and offload tasks from the model.

And if you realize that you are using LLMs repeatedly for the same task, ask the LLM to build a script instead of doing the work directly and then tell it, via instructions, to use that script in the future.

### Consider when to start a new conversation

Long conversations can benefit from caching, but they also keep growing: each new turn may require more previous context to be re-processed. For related tasks, staying in one conversation can make sense. For independent tasks, splitting them into separate conversations may reduce re-processed tokens.

As a rule of thumb: start a new conversation when tasks are mostly independent, or when you return after a longer break and the cache may have expired.

### Avoid switching models mid-conversation

Changing the model can reset or reduce the benefit of cached tokens. If the task does not require a different model, stay with the same one to avoid paying again for context that was already processed.

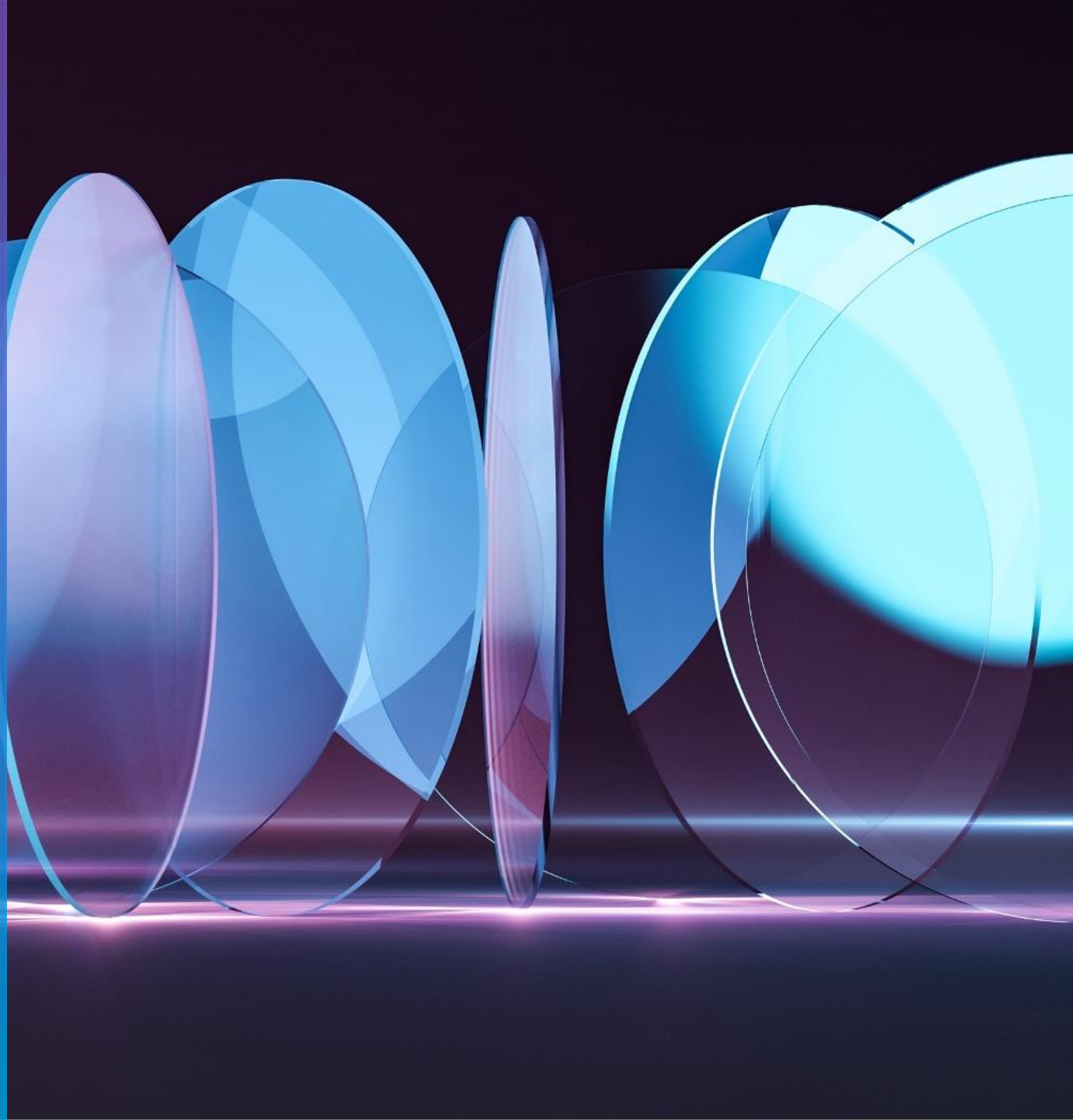
### Toggle tools intentionally

Enabled tools add context to the conversation and can increase token usage. Keep only the tools active that are needed for the current task, and turn off anything unrelated.

### Progressive disclosure

Don't front-load everything into the context. Start with lightweight pointers – file names, headings, skill descriptions – and let the agent pull the full detail only when a task actually needs it. Context stays small, and the model keeps its attention on what matters.

# Scaling AI across Teams





# How can we share artefacts across repos and teams?

AGENTS.md · March

```
  # Coding rules
  Write tests for every change.
- Never log secrets.
  Use conventional commits.
```

Still the version in the Payments and Orders repos

AGENTS.md · May

```
  # Coding rules
  Write tests for every change.
+ Never log secrets, tokens or personal data.
  Use conventional commits.
```

Updated in the Web shop repo only

Public

# Artefacts reach agents by location, reference or package

**Found by location** 1
Picked up automatically

**Referenced by path** 2
One file points to another

**Shipped as a package** 3
Versioned – needed once repos multiply

| Technique | Claude Code | GitHub Copilot |
|---|---|---|
| **AGENTS.md + skills folder**<br>Open formats both agents read | `.claude/skills/`<br>AGENTS.md via a pointer (see below) | `AGENTS.md` `.claude/skills/`<br>both read natively |
| **Path-scoped rules**<br>Extra rules for one folder only | `.claude/rules/*.md` `paths:` | `*.instructions.md` `applyTo:` |
| **Parent repository**<br>A nested repo inherits the outer setup | built in, no setting needed | `chat.useCustomizationsInParentRepositories` |
| **User folder**<br>Personal rules, skills and agents | `~/.claude/skills/`<br>`~/.claude/agents/` | `~/.copilot/skills/` `~/.copilot/agents/` |
| **Symlinks and imports**<br>Bridge to an agent's own file name | `CLAUDE.md:` `@AGENTS.md`<br>or symlink CLAUDE.md → AGENTS.md | not needed – reads AGENTS.md directly |
| **Explicit paths**<br>Pull in a folder outside the repo | `@../shared-standards/AGENTS.md` | `chat.instructionsFilesLocations` |
| **Git submodule**<br>Pinned artefact repo – any stack | `@standards/AGENTS.md` | `chat.instructionsFilesLocations` |
| **Package (npm, …)**<br>Dependency – one ecosystem only | `@node_modules/acme-rules/AGENTS.md` | `chat.instructionsFilesLocations` |
| **Marketplace / org settings**<br>Bundles skills, agents, hooks, MCP | `extraKnownMarketplaces` | Org instructions and custom agents |

© Zühlke 2026

Public

# Across repos, shared artefacts need a versioned package

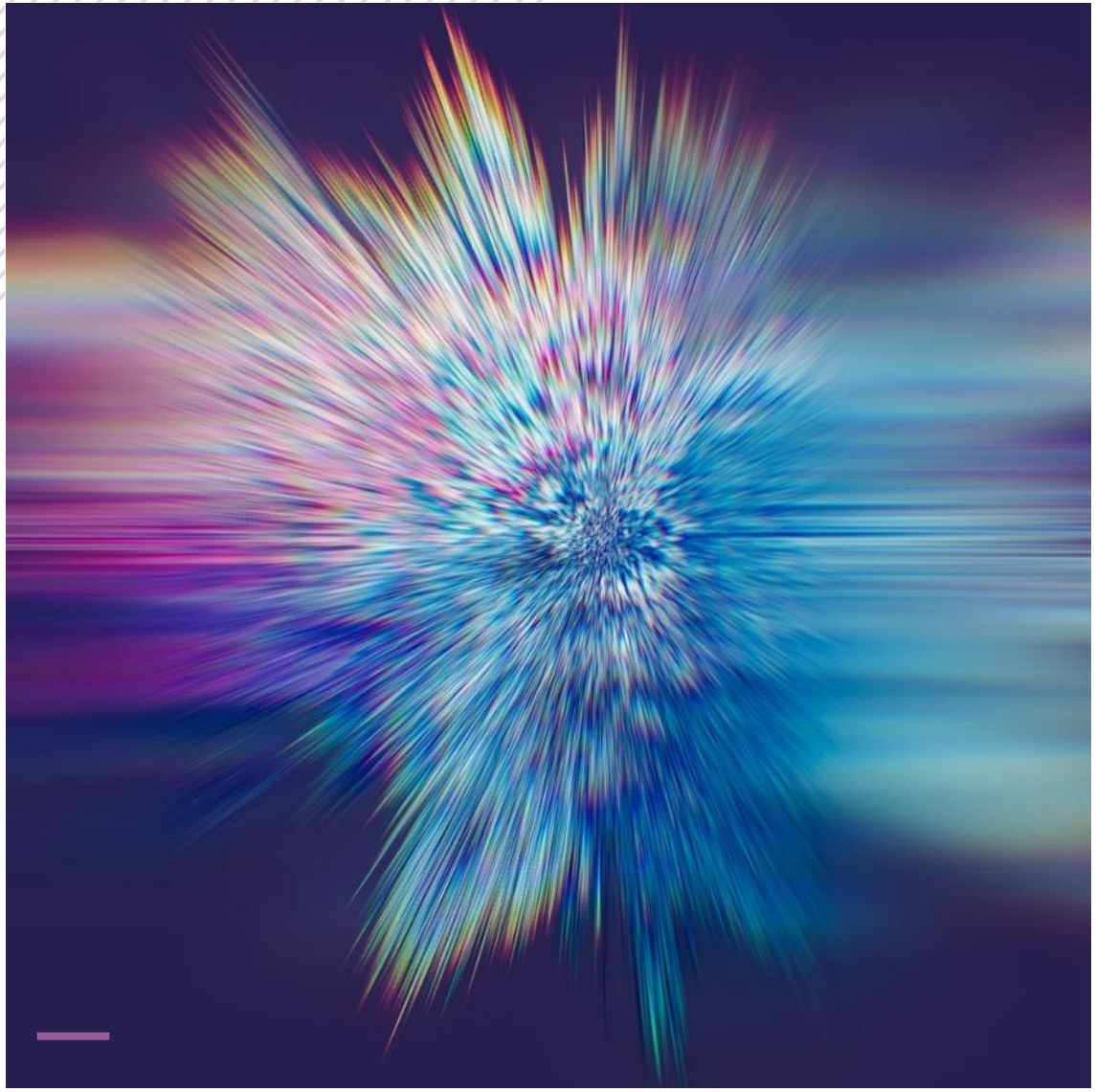
| | **Monorepo**<br>The repo itself distributes the artefacts | **Multiple repos**<br>Artefacts must travel between repos |
|---|---|---|
| **Only you**<br>Personal preferences and helpers<br>e.g. your own review skill | **User folder**<br>Lives in ~/.claude/ or ~/.copilot/ and applies in every repo you open. Nobody else sees it – fine for preferences, not for team standards. | |
| **One team**<br>Conventions of one team or service<br>e.g. payments rules and test skills | **Path-scoped rules + skills folder**<br>The team's rules apply only to its folder, its skills sit next to the code. Every change goes through the normal pull request. | **Git submodule or package**<br>The artefacts live in their own repo and every team repo pins a version. An update is an explicit, reviewable version bump.<br>Note: Explicit paths work too, but break silently if one folder is laid out differently |
| **Several teams**<br>Company-wide standards<br>e.g. security rules and skills | **Root AGENTS.md + skills folder**<br>Shared artefacts sit at the root and reach every team automatically. Each team adds path-scoped rules for its own folder. | **Plugin marketplace or org settings**<br>Published once by an owner, installed on every machine – Claude Code via managed settings, Copilot via GitHub org settings.<br>Note: Submodules work too – language packages only if all teams share a stack |

© Zühlke 2026

# Applying AI to brownfield projects



Public



**Be cautious: AI amplifies every pattern – good and bad.**

Using AI alone does not guarantee success. Good practices scale faster – but so do the bad ones: technical debt, code smells, missing tests and undocumented decisions.

The effect is strongest in brown-field projects: AI learns from the code base it is given, so existing weaknesses spread with every generated change.

Related study: 2025 DORA Report | Google Cloud

Public

# Strategies for dealing with brownfield projects



### Make implicit knowledge explicit

- Derive specs and ADRs from code and SME interviews
- Add missing knowledge, e.g. via a knowledge layer
- Use progressive disclosure to not overwhelm the context with information



### Refactor for AI agents

- Slice the codebase vertically with deep modules and clear interfaces
- Improve test quality and coverage
- Automate and add more quality checks

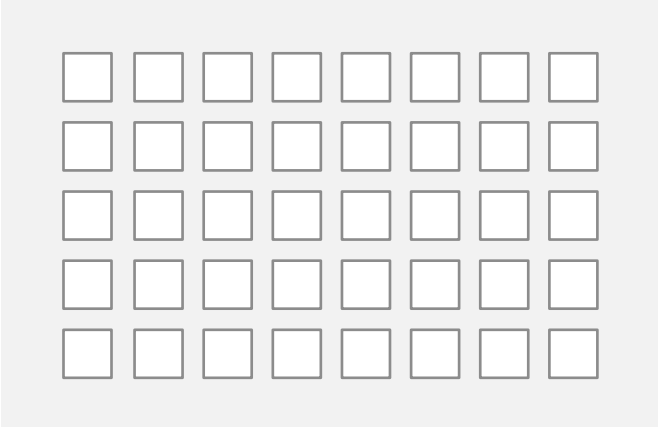


### Change incrementally

- Try to get rid of bad patterns step by step
- Use an incremental, change-based approach
- Just as before AI: start small and build up the necessary assets one change at a time

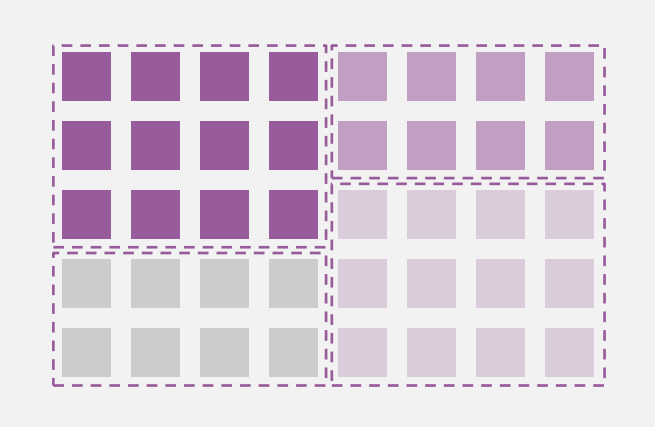
Public

# Deep modules bake our intent into the code structure, so AI no longer has to rebuild context from scratch



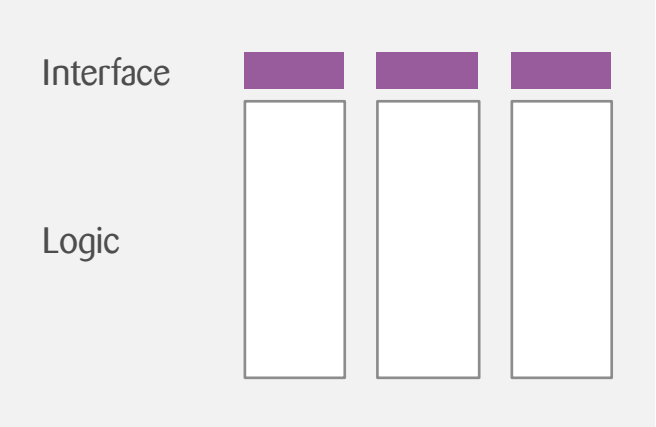
## 1) How AI sees your code

To an AI agent, a codebase is a flat grid of files and functions. **Nothing tells it which parts belong together.**



## 2) How we see it

We know which parts form a unit. AI has to **reconstruct these relations in every session**, which costs context and leads to wrong assumptions.




## 3) Structure code around modules

Deep modules have small interfaces hide a lot of logic. AI **only needs to know the interface**, and tests can run at the module boundaries.

Public

# Progressive disclosure allows agents to explore large code-bases in a token-efficient way

**Project structure**

```
repo/
  AGENTS.md
  knowledge/
    INDEX.md
    craft/          writing techniques
      INDEX.md      next level down
    voice/          9 tone archetypes
    platforms/      rules, templates
    posts/          performance data
    hypotheses/     what to test next
```

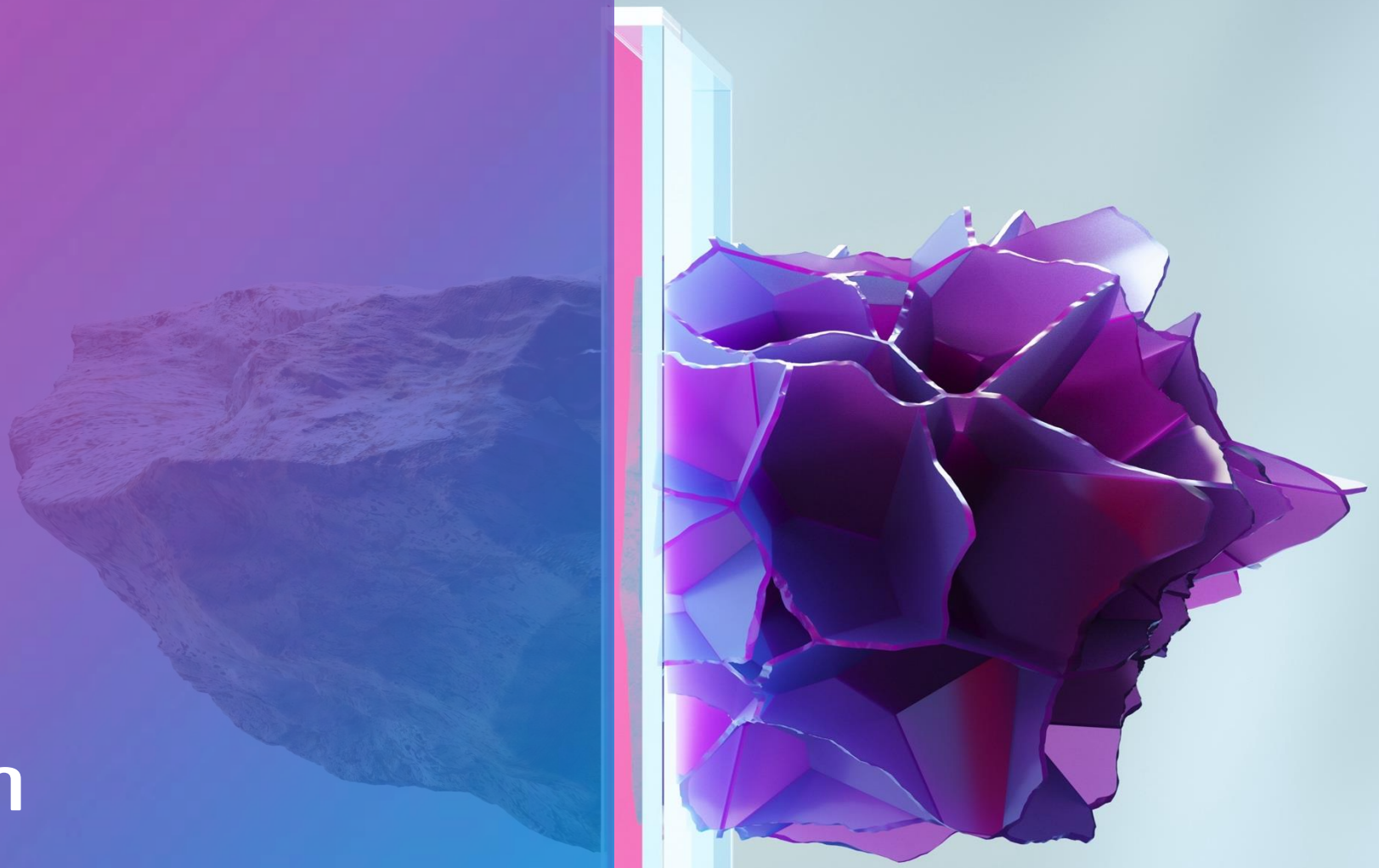
**AGENTS.md · read every session**

```
# AGENTS.md
## Navigation
→ - Start at knowledge/INDEX.md
→ - In every folder, read its INDEX.md before any other file
- Open only the files the task needs
## Learning mode
- Log results in posts/, test ideas in hypotheses/
- Propose edits to this knowledge base
```

**knowledge/INDEX.md · read first, then one folder**

```
# Knowledge index
craft/       writing techniques   → open craft/INDEX.md next
voice/       9 tone archetypes    → setting the tone
platforms/   rules, templates     → formatting for a channel
posts/       performance data     → checking what performed
hypotheses/  what to test next    → running a test
```

# Adoption



Public

# Developers spend only 14% day writing code[1] – the real value sits in the other 86% across the full software development lifecycle

**Maintenance**

Support and operate the deployed software

**Planning**

Define project scope, objectives, and resources

**Testing & Integration**

Verify functionality, identify defects, and integrate components

**Analysis**

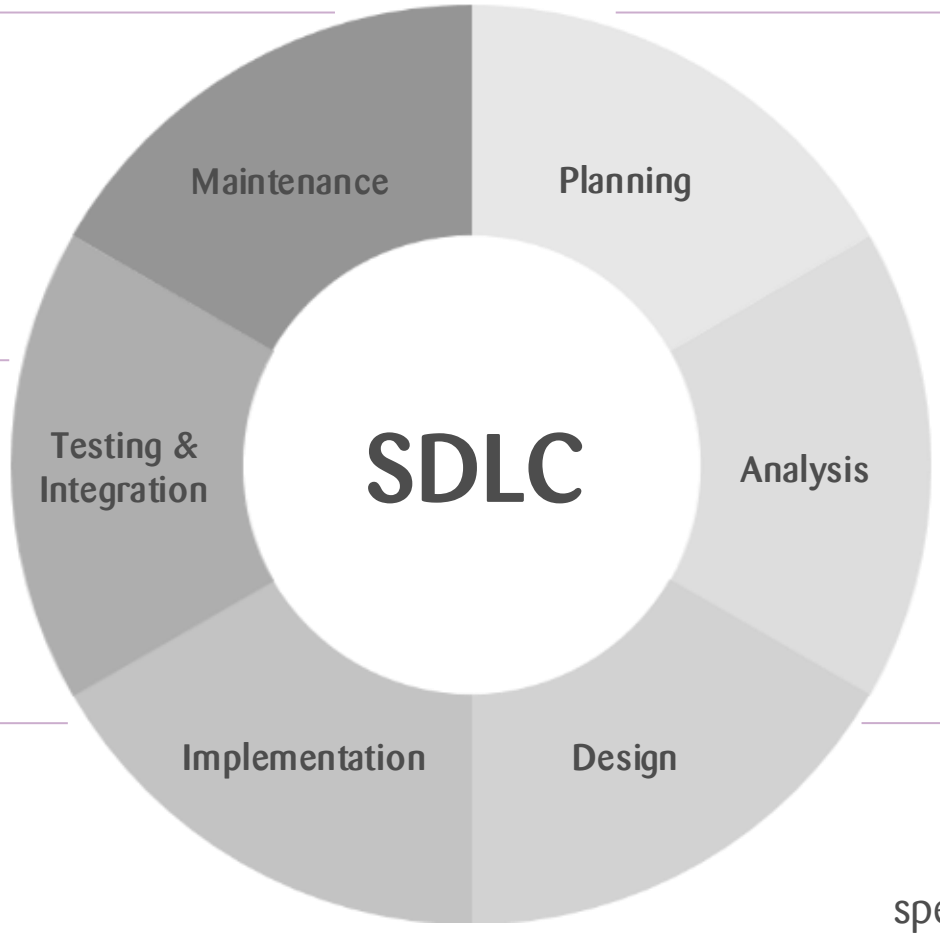
Gather and analyse requirements from stakeholders

**Implementation**

Develop and code the software

**Design**

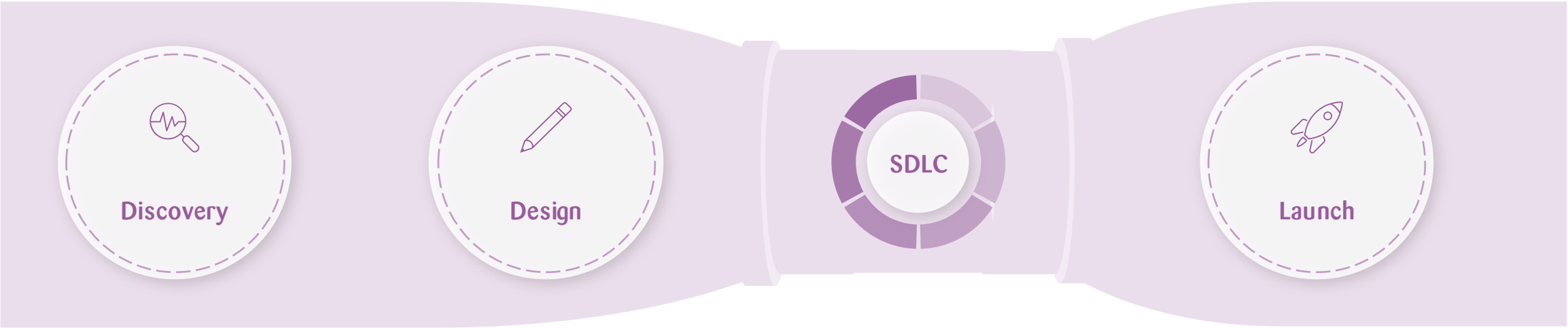
Create system architecture, technical specifications, and user interface designs




[1] https://www.microsoft.com/en-us/research/wp-content/uploads/2024/11/Time-Warp-Developer-Productivity-Study.pdf

Public

# Accelerating application development alone pushes the bottleneck upstream and downstream; to leverage the full potential AI needs to be thought across teams




**Value Stream Mapping** makes the full path from idea to delivered value visible – every step, handoff, queue, and approval. By separating time spent *working* from time spent *waiting*, it locates the actual constraint rather than the phase that merely looks busiest. Applied across the whole PDLC, not to a single stage.

Public

# Start with optimization levers for early returns, but the real leverage lies in transformation

## Impact on efficiency

Optimisation levers improve existing workflows by using AI assistants to automate tasks in individual disciplines without changing underlying practices.

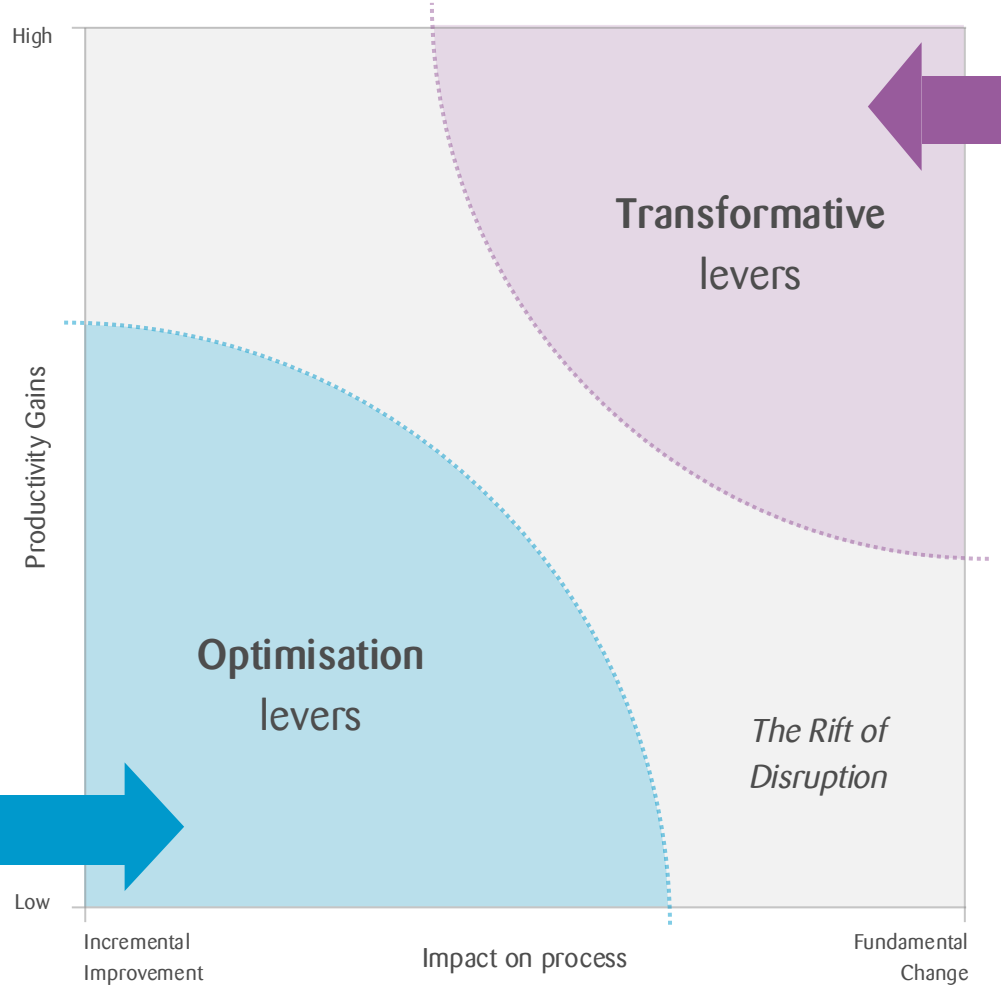
PROS
- Easy to implement, less disruption, lower barrier to entry, fast ROI.

CONS
- Offer only incremental gains and limited long-term innovation.

**Productivity impact:** Low to moderate productivity gains from faster task completion, but still effective at scale.

**Recommendations:** Use the incremental improvements to focus on higher value work in the problem and solution space and to introduce transformative patterns.

Productivity gains: low to moderate




Productivity gains: medium to high

## Impact on effectiveness

Transformative levers use AI augmentation to restructure workflows across disciplines and fundamentally change delivery practices.

PROS
- New capabilities, high scalability and better competitive differentiation.

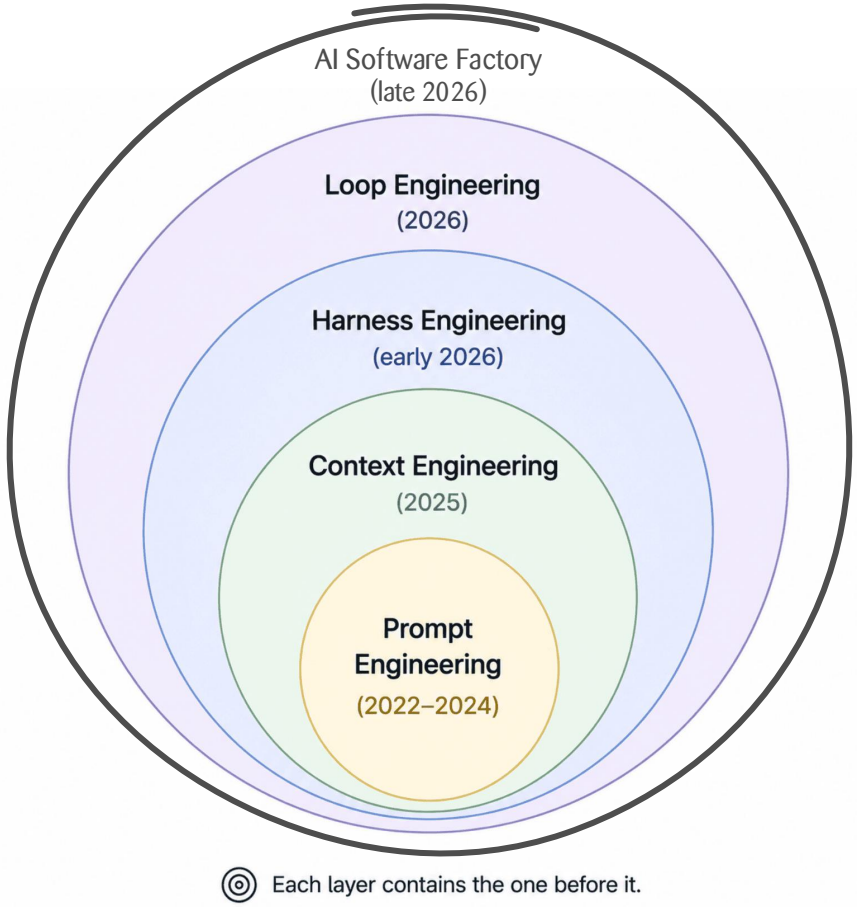
CONS
- Require process adaptation, higher effort to address change resistance.

**Productivity impact:** Medium to high productivity gains from fully autonomous AI-driven optimisation.

**Recommendations:** Implement based on experience with Optimisation Patterns to fundamentally rethink engineering and business processes.

Public

# What "using AI" in software engineering means changes annually - teams need to find what fits them and remain willing to change it as they learn more




*Source: https://machinelearningmastery.com/an-introduction-to-loop-engineering/*

Public

# It takes some time to develop a good intuition. Use the formats and communities within Zühlke to exchange experiences, share knowledge, and learn from each other



### ”How I use AI” Sessions

A peer-to-peer community series at Zühlke. Each session, a colleague walks through how they use AI in their day-to-day work, with concrete tools and live demos rather than slideware. Register for upcoming sessions in Workday.



### Courses & Trainings

The AI Education Catalogue offers curated AI learning opportunities across formats, from live classrooms and online sessions to self-paced courses, videos, LinkedIn Learning, and mandatory training, tailored to audiences such as everyone, management, change managers, software engineering, cybersecurity, UX, sales, and AI product builders.



### Community of Practice

The Cybernetic Delivery Community of Practice at Zühlke Switzerland helps teams adopt AI in software delivery through practical use cases, validated learnings, and reusable patterns across the SDLC. Join them via Link.


zühlke
empowering ideas